

Z-score와 IsolationForest

정상만 배워서 이상을 찾는 두 가지 방법, 그리고 각자의 한계

평균 · 표준편차 · Z-score 임계값과 트레이드오프

MIMII 소리 특징 세 개 다변량 이상 고립 · contamination

2026년 9월 9일 (수) · 36일차 · 79차시

교안 25_03 후반 · 26_01 Z-score 이상탐지 해석 · 26_02 IsolationForest 이상탐지 · 26_03 이상 라벨링 초반

국립금오공과대학교 컴퓨터공학부 박민호

교안 / PART	다룬 범위	상태
26_01 / 01	이상탐지와 Z-score 기초 — 기준선 · 임계값 · 트레이드오프	완주
26_01 / 02	MIMII 데이터 이해 — 소리 특징 세 개와 label	완주
26_01 / 03	Z-score 적용과 해석 — 실습 3~7	완주
26_01 / 04	Z-score의 한계 — 한 특징 · 정규분포 · 오염	완주
26_02 / 01~03	IsolationForest 원리 · 적용 · 파라미터	완주
26_03 / 01	이상 라벨링 — 라벨 변환의 필요성	초반만
26_03 / 02~04	산점도 · 기준선 · 정비 판단 연결	다음 시간

이 책을 읽는 법

새로 넣은 그림 · 배지 · 오늘의 위치

1. 이번 문서부터 그림이 들어갑니다

이번 회차부터 본문에 차트를 넣었습니다. 오늘 배운 내용은 "정상은 좁게 모이고 이상은 흩어진다" 같은 **공간적인 이야기**가 많은데, 이런 것은 글로 열 줄 읽는 것보다 그림 한 장을 보는 편이 훨씬 빠릅니다.

그림은 **전부 오늘 실습 데이터로 직접 그린 것**입니다. 교안에 있는 그림을 옮긴 것이 아니라, 26_mimii_features.csv 220행을 실제로 읽어서 계산하고 그렸습니다. 따라서 그림에 찍힌 점 하나하나가 실습 데이터의 실제 행이고, 그림에 적힌 숫자는 본문 표의 숫자와 정확히 같습니다.

그림 종류	언제 나오나	무엇을 보라는 뜻인가
히스토그램	분포를 이야기할 때	값들이 어디에 몰려 있고 어디가 비어 있는가
산점도	두 특징의 관계를 볼 때	한 축에서 겹치는 점이 다른 축에서 갈라지는가
막대그래프	설정을 바꿔 비교할 때	설정 하나를 바꾸면 결과가 얼마나 달라지는가
개념도	알고리즘 원리를 설명할 때	계산 과정을 그림으로 옮기면 이렇게 생겼다

개념

그림 아래 회색 한 줄이 캡션입니다. 그림에서 무엇을 봐야 하는지를 한 문장으로 적어 두었으니, 그림을 먼저 훑고 캡션을 읽은 뒤 본문으로 돌아오시면 됩니다. 그림만 따로 봐도 흐름이 이어지도록 순서를 잡았습니다.

2. 처음 나오는 도구는 반드시 먼저 설명합니다

오늘은 **scikit-learn**이라는 새 라이브러리가 등장합니다. 지금까지 쓰던 pandas·numpy·matplotlib·seaborn과는 성격이 다른 도구라, 갑자기 `iso.fit(X)` 같은 코드가 나오면 막힐 수 있습니다.

그래서 이 문서는 **새 함수·메서드가 처음 나오는 자리마다 그것부터 설명**하고 넘어갑니다. 예를 들어 `fit()`이 무엇인지 모르는 상태에서 `iso.fit(X)`를 보여 주지 않고, "**모델을 만들고 → fit으로 학습시키고 → predict로 예측한다**"는 세 단계를 먼저 그림과 함께 설명한 뒤에 코드를 보여 드립니다.

오늘 처음 나오는 도구	무엇인가	어디서 설명하나
sklearn (scikit-learn)	파이썬 머신러닝 라이브러리	CH 5-6

오늘 처음 나오는 도구	무엇인가	어디서 설명하나
IsolationForest	이상탐지 모델 — 여러 특징을 동시에 봄	CH 5-1 ~ 5-5
.fit()	모델에게 데이터를 보여 주고 학습시키는 메서드	CH 5-6
.predict()	학습된 모델로 판정하는 메서드 — 결과가 -1과 1	CH 6-1
.score_samples()	이상 점수를 숫자로 돌려주는 메서드	CH 6-4
StandardScaler	특징 범위를 맞추는 전처리 도구	CH 7-3
pd.crosstab()	두 컬럼을 교차해 표로 세는 함수	CH 6-3

주의

그래도 모르는 것이 나오면 그 자리에서 멈추셔도 됩니다. 이 문서는 앞에서 설명한 것만 뒤에서 쓰도록 순서를 짰습니다. 만약 어떤 코드가 갑자기 이해가 안 된다면, 그 도구가 처음 나온 절로 돌아가 보시면 됩니다 — 절 번호를 본문에 함께 적어 두었습니다.

3. 항목 뱃지 읽는 법

개념 이상탐지·통계 도메인 개념. 코드가 아니라 생각하는 방법입니다.

함수 라이브러리가 제공하는 함수입니다. 앞에 점이 없습니다. `pd.crosstab()` 처럼.

메서드 객체 뒤에 점을 찍고 괄호를 붙여 호출합니다. `iso.fit()` 처럼.

속성 객체 뒤에 점을 찍되 괄호가 없습니다. `df.shape` 처럼.

패턴 여러 도구를 엮어 쓰는 **관용 표현**입니다.

개념

3초 판별법. ① 앞에 점(.)이 있나? 없으면 함수입니다. ② 점이 있으면 괄호()가 붙나? 붙으면 **메서드**, 안 붙으면 **속성**입니다. 오늘 나오는 `iso.fit(X)`는 점도 있고 괄호도 있으니 메서드이고, `df.shape`는 점은 있고 괄호가 없으니 속성입니다.

4. 오늘의 위치 — 지금까지 배운 것 위에

오늘 배우는 것은 **완전히 새로운 것이 아닙니다**. 20~23일차에 배운 시계열·통계 도구 위에, 25일차에 배운 머신러닝 최소 개념을 얹은 것입니다.

회차	거기서 배운 것	오늘 어떻게 쓰이나
16일차	이상치 개념과 사분위수 · IQR	같은 "이상치"를 이번엔 Z-score로
17·18일차	히스토그램 · 산점도 · seaborn	오늘 그림 전부 — hue로 색 나누기까지

회차	거기서 배운 것	오늘 어떻게 쓰이나
21일차	평균 · 표준편차 · 변동성	Z-score 계산식의 재료 그 자체
25_01	머신러닝 최소 개념 — 지도/비지도	이상탐지가 왜 비지도인가
25_02	이상탐지와 통계 기초	정상 기준선을 세우는 발상
25_03	Z-score 기반 이상치 탐색	오늘 오전에 이어서 마무리

연결

16일차의 IQR과 오늘의 Z-score는 같은 목적, 다른 방법입니다. IQR은 사분위수(중앙값 근처의 순위)로 이상치를 찾고, Z-score는 평균·표준편차(중심과 흩어짐)로 찾습니다. 둘 다 **한 컬럼만 보는 1차원 방법**이라는 점도 같고, 그래서 오늘 CH 4에서 지적하는 한계도 똑같이 적용됩니다.

CONTENTS

목차

오늘 문서의 지도

장	제목	무엇을 배우나
CH 0	오늘의 지도	오늘의 데이터 · 오늘 배우는 두 도구의 관계
CH 1	이상탐지와 Z-score 기초	기준선 · 계산식 · 임계값 · 두 가지 실수의 트레이드오프
CH 2	MIMII 데이터 이해	소리 특징 세 개(세기·톤·거칠기)와 label의 역할
CH 3	Z-score 적용과 해석	기준선 세우기 → 계산 → 판정, 그리고 정비 문장으로 번역
CH 4	Z-score의 한계	한 특징만 본다 · 정규분포 가정 · 평균이 흔들린다
CH 5	IsolationForest 원리	고립이라는 발상 · 분할 횟수 · contamination
CH 6	모델 적용과 결과 해석	<code>fit</code> · <code>predict</code> · <code>score_samples</code> · 교차표 · Top N
CH 7	파라미터와 활용	contamination 실험 · 스케일링 · 새 데이터 적용 · 두 방법 비교
CH 8	이상 라벨링 (초반)	-1/1을 우리 형식으로 바꾸는 일과 그 이유
부록 A	치트시트	오늘 나온 숫자와 코드를 한 장으로
부록 B	용어·오류 사전	헛갈리는 용어 · 교안과 실제 결과의 차이
부록 C	하이라이트와 다음 시간	오늘의 핵심 발견 · 다음 진도

CH 1~4가 **교안 26_01**(61쪽), CH 5~7이 **26_02**(59쪽), CH 8이 **26_03**(68쪽 중 앞부분)입니다. 분량이 크지만 **CH 1~4와 CH 5~7이 같은 구조**로 되어 있어서, 한 번 흐름을 잡으면 두 번째는 훨씬 빠르게 읽힙니다 — 원리 → 적용 → 한계의 순서가 똑같습니다.

주의

오늘 STT는 **296분(10세그먼트)**으로 하루 전체가 담겨 있습니다. 세그먼트별 주제가 교안 순서와 맞아떨어져 진도 확정에 문제가 없었습니다 — seg1~4가 Z-score, seg6~9가 IsolationForest, seg10이 이상 라벨링입니다. 다만 **seg2·seg9에는 취업·진로 상담 이야기가 섞여** 있는데, 수업 내용이 아니라 문서에서 제외했습니다.

오늘의 지도

정상만 배워서 이상을 찾는다는 발상

0-1. 이상탐지가 하려는 일

이상탐지란 **고장이 난 뒤가 아니라, 평소와 달라지는 순간을 먼저 알아채는 일**입니다. 교안이 이것을 세 가지로 나눠 설명합니다.

구분	내용	풀어 쓰면
시점	고장 후가 아니라 평소와 달라지는 순간	설비가 멈춘 뒤에 알면 늦습니다. 멈추기 전에 "어라, 오늘 소리가 좀 다른데" 하는 순간을 잡자는 것입니다.
목적	예측 정비(PdM)의 출발점	징후를 미리 잡아야 라인 정지와 비용 폭증을 막을 수 있습니다.
역할	사람을 대체하지 않는 보조 눈	24시간 감시를 사람이 계속 할 수는 없으니, 자동으로 훑어 주고 사람이 확인하는 구조입니다.

개념

"보조 눈"이라는 표현이 오늘 전체를 관통합니다. 이상탐지 모델이 내놓는 것은 "고장입니다"가 아니라 "**평소와 다르니 살펴볼 만합니다**"입니다. 이 차이가 CH 3-6절의 정비 문장 작성과 CH 8의 라벨링 이야기까지 이어집니다.

0-2. 두 기둥 — 기준과 벗어남

이상을 찾으려면 먼저 **정상이 무엇인지**를 정해야 합니다. 교안이 든 비유가 아주 직관적입니다.

기둥	내용	지하철 비유
① 기준	평소의 중심값과 흔들림 폭	평소 8시 지하철을 탄다 — 이게 중심값
② 벗어남	기준에서 크게 떨어진 값을 이상으로 의심	8시 2분은 정상 흔들림, 8시 30분은 이상

여기서 중요한 것은 **8시 2분과 8시 30분을 가르는 기준이 "평소 얼마나 흔들리느냐"에 달려 있다**는 점입니다. 매일 7시 55분~8시 5분 사이에 타는 사람에게 8시 30분은 명백한 이상이지만, 매일 7시 30분~8시 30분 사이 아무 때나 타는 사람에게는 평범한 날입니다. **같은 30분 차이가 사람에 따라 이상이 되기도 하고 정상이 되기도 합니다.**

개념

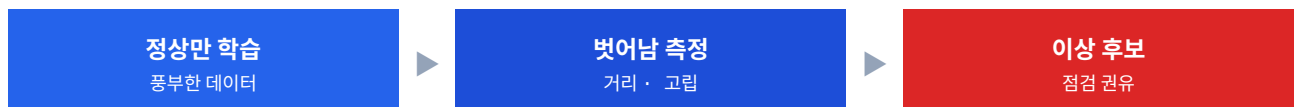
그래서 평균 하나만으로는 부족하고 흔들림 폭이 반드시 함께 필요합니다. 이 두 숫자가 앞으로 계속 나올 평균 (mean)과 표준편차(std)이고, 둘을 묶어 하나의 점수로 만든 것이 **Z-score**입니다. CH 1에서 자세히 다룹니다.

0-3. 정답 없이 찾는다 — 비지도학습

25일차에 지도학습과 비지도학습을 배웠습니다. 이상탐지는 **비지도학습**입니다. 왜 그런지 교안이 세 질문으로 정리합니다.

질문	답
지도학습이란	입력과 정답을 함께 주고 그 관계를 배우게 하는 방식
비지도학습이란	정답 없이 데이터 속 구조·패턴만으로 찾아내는 방식
이상탐지는 왜 비지도인가	고장은 드물고 종류도 제각각이라 이상 정답표 확보가 사실상 불가능하기 때문

세 번째가 핵심입니다. 베어링 마모·누유·회전 불균형처럼 **고장 종류가 워낙 다양해서** 모든 이상에 라벨을 붙이는 것이 현실적으로 불가능합니다. 그래서 발상을 뒤집습니다.



개념

교안의 비유가 좋습니다 — **"우리 집 식구 얼굴을 알면 낯선 사람도 바로 알아챈다."** 도둑의 얼굴을 미리 외울 필요가 없습니다. 식구 얼굴만 정확히 알면 **식구가 아닌 것은 전부 걸러집니다**. 그래서 **정상을 얼마나 충실히 학습했느냐가 성능을 좌우합니다**.

주의

이 발상에는 **전제가 하나** 숨어 있습니다 — **정상 데이터가 진짜 정상이어야** 합니다. 식구 사진 속에 도둑이 한 명 섞여 있으면 그 도둑은 영원히 못 잡습니다. CH 4-4절에서 이 문제를 "기준선 오염"이라는 이름으로 다시 다룹니다.

0-4. 오늘 다루는 데이터

오늘 실습은 **MIMII**라는 공개 데이터를 씁니다. 일본 히타치 연구진이 2019년에 공개한 **산업 기계 소리 데이터셋**입니다. 다만 우리는 소리 원본(wav 파일)이 아니라 **거기서 뽑아낸 특징값 CSV**를 다룹니다.

파일	크기	들어 있는 것	쓰는 곳
26_mimii_features.csv	220행 4열	특징 3개 + label	CH 1~7 전부

파일	크기	들어 있는 것	쓰는 곳
26_mimii_new.csv	40행 3열	특징 3개 · label 없음	CH 7-4 새 데이터 적용

첫 확인 — 교안 실습 1

```
import pandas as pd
```

```
df = pd.read_csv("26_mimii_features.csv")
new = pd.read_csv("26_mimii_new.csv")
```

```
print("features:", df.shape, list(df.columns))
print("new      :", new.shape, list(new.columns))
print()
print(df["label"].value_counts().sort_index().to_string())
```

```

> features: (220, 4) ['rms', 'spectral_centroid', 'zero_crossing_rate', 'label']
> new      : (40, 3) ['rms', 'spectral_centroid', 'zero_crossing_rate']
>
> label
> 0      180
> 1       40

```

개념

정상 180개, 이상 40개입니다. 이상이 전체의 약 18%인데, 이 비율이 나중에 IsolationForest의 `contamination=0.18`이라는 설정값의 근거가 됩니다(CH 5-8절). 그리고 26_mimii_new.csv에 label 컬럼이 없다는 점이 중요합니다 — 실제 현장이 그렇기 때문에 일부러 없앤 것입니다.

0-5. 오늘 배우는 두 도구의 관계

오늘 배우는 Z-score와 IsolationForest는 **경쟁 관계가 아닙니다**. 교안이 여러 번 강조하듯 **보는 방식이 다른 동료**입니다.

항목		
보는 방식	특징을 하나씩 따로	여러 특징을 한꺼번에
판정 근거	평균에서 표준편차 몇 칸 떨어졌나	몇 번 잘라야 혼자가 되나
필요한 것	정상 데이터의 평균·표준편차	특징들과 이상 비율 추정치
강점	직관적이고 설명하기 쉬움	특징 사이 관계가 어긋난 이상을 잡음
약점	조합이 이상한 경우를 원리적으로 못 잡음	설정값 부담 · 왜 그렇게 판정했는지 설명이 어려움

개념

오늘의 이야기 구조가 여기서 정해집니다. CH 1~3에서 Z-score를 배우고, CH 4에서 그것이 무엇을 못 하는지 확인하고, 그 못 하는 것을 하기 위해 CH 5에서 IsolationForest로 넘어갑니다. **한계를 먼저 알아야 다음 도구가 왜 필요한지 이해됩니다** — 교안이 이 순서를 의도적으로 짰습니다.

0-6. 오늘 처음 나오는 용어 열 개

용어	한 줄 뜻	어디서
기준선	정상 데이터로 계산한 평균과 표준편차 — 비교의 출발점	CH 1-2
Z-score	평균에서 표준편차 몇 칸 떨어졌나를 나타내는 점수	CH 1-3
임계값	이 선을 넘으면 이상이라고 정한 경계 — 사람이 정함	CH 1-5
거짓 경보 · 놓침	정상인데 이상이라 함 / 이상인데 정상이라 함	CH 1-6
특징값	복잡한 신호를 몇 개의 대표 숫자로 요약한 것	CH 2-1
다변량 이상	각 특징은 정상인데 조합이 비정상인 이상	CH 4-2
고립 (Isolation)	분할을 반복해 한 점만 남기는 것 — 이상은 빨리 고립됨	CH 5-2
contamination	전체 중 이상이 차지하는 대략의 비율 (오염도)	CH 5-8
이상 점수	얼마나 이상한지의 정도 — 순위를 매길 수 있게 해 줌	CH 6-4
회색 지대	정상과 이상이 겹쳐 어떤 도구도 헛갈리는 구간	CH 7-6

이상탐지와 Z-score 기초

평균에서 몇 칸 떨어졌나를 하나의 점수로

1-1. 평균은 중심, 표준편차는 흩어짐

Z-score를 이해하려면 먼저 **평균과 표준편차가 각각 무엇을 말하는 숫자인지**를 감으로 잡아야 합니다. 수식보다 느낌이 먼저입니다.

숫자	뜻	한 줄로
평균	데이터가 모이는 중심	평소 값들이 모여 있는 가운데 자리
표준편차	중심에서 평소 흩어지는 폭	평소 어느 정도까지 들쭉이는지의 크기

교안이 든 예시가 이 둘이 왜 **항상 짝**이어야 하는지를 잘 보여 줍니다. **두 반의 평균이 똑같이 70점**이라고 해 봅시다. 그런데 한 반은 점수가 65~75점 사이에 몰려 있고, 다른 반은 30~100점으로 흩어져 있습니다. 평균만 보면 두 반이 같지만 **완전히 다른 반**입니다.

개념

여기서 **"85점을 받은 학생이 특이한가"**를 물으면 답이 갈립니다. 첫 번째 반(65~75)에서는 85점이 아주 튀는 점수지만, 두 번째 반(30~100)에서는 흔한 점수입니다. **같은 85점인데 이상 여부가 반대**입니다. 이것이 표준편차가 반드시 필요한 이유입니다 — **얼마나 벗어나야 이상인지**는 평소 흔들림 폭이 정합니다.

1-2. 기준선은 정상 데이터로만 만듭니다

평균과 표준편차를 묶어 **기준선**이라고 부릅니다. 그런데 이 기준선을 만들 때 **반드시 지켜야 할 원칙**이 하나 있습니다.

기준선은 정상 데이터로만 만든다

기둥	내용	안 지키면
평균	정상일 때 데이터가 모이는 중심값	평균이 이상 쪽으로 끌려감
표준편차	중심에서 평소 흔들리는 폭	폭이 부풀어 넓어짐
정상만 사용	이상이 섞이면 기준이 한쪽으로 끌려가 오염	넓어진 정상 범위 안에 이상이 숨어 안 잡힘

주의

이것이 이 단원에서 가장 흔한 실수입니다. 코드로 쓰면 `df["rms"].mean()`이 아니라 `df[df.label==0][ "rms" ].mean()` 이어야 합니다. 한 글자 차이지만 결과는 완전히 달라집니다 — CH 4-4절에서 숫자로 확인합니다.

실습에서는 `label` 컬럼이 있으니 정상을 고르기 쉽습니다. 하지만 **현장에는 정답이 없습니다**. 그럴 때는 이렇게 합니다.

단계	현장에서 하는 일
① 정상으로 간주	새 설비이거나 정상이 확실한 초기 운전 기간을 정상으로 사용
② 정비 기록 점검	그 기간에 고장·이상 신호가 없었는지 확인
③ 원리는 동일	깨끗한 정상으로 기준을 잡는다는 원리는 같고, <code>label</code> 대신 "믿을 수 있는 기간"이라는 점만 다름

연결

33일차의 라벨 설계와 정확히 같은 이야기입니다. 그때도 "정상 구간을 어디로 볼 것인가"를 정비 이력으로 결정했습니다. 오늘은 그 정상 구간으로 **기준선을 계산**한다는 점만 추가됩니다.

1-3. Z-score — 몇 칸 떨어졌나를 세는 점수

이제 본론입니다. 평균과 표준편차가 준비됐다면, 어떤 값이 **평균에서 표준편차 몇 칸 떨어져 있는지**를 계산할 수 있습니다. 그 칸 수가 Z-score입니다.

$$Z = (\text{값} - \text{평균}) \div \text{표준편차}$$

식을 두 조각으로 나눠 읽으면 뜻이 분명해집니다. **먼저 빼서 중심으로부터의 거리를 구하고, 그다음 나눠서 그 거리를 "평소 폭 몇 개분"으로 환산**합니다.

조각	하는 일	왜 필요한가
값 - 평균	중심에서 얼마나 떨어졌는지 거리를 구함	절대적인 차이
÷ 표준편차	그 거리를 평소 흔들림 폭 단위로 환산	평소 폭이 큰 값과 작은 값을 같은 잣대로 비교

구체적인 숫자로 확인해 보겠습니다. 오늘 데이터에서 정상 `rms`의 평균은 **0.050**, 표준편차는 **0.012**입니다.

Z-score 손계산

정상 데이터로만 평균·표준편차 계산 (1-2절 원칙)

```
normal = df[df["label"] == 0]
```

```
mu = normal["rms"].mean()
```

```
sd = normal["rms"].std()
```

```
print(f"정상 평균 {mu:.4f} · 정상 표준편차 {sd:.4f}")
```

값 하나를 손으로 확인

```
for v in [0.060, 0.074, 0.120]:
```

```
    print(f"값 {v} → ({v} - {mu:.3f}) / {sd:.3f} = Z {(v-mu)/sd:5.2f}")
```

▶ 정상 평균 0.0500 · 정상 표준편차 0.0122

▶ 값 0.06 → (0.06 - 0.050) / 0.012 = Z 0.82

▶ 값 0.074 → (0.074 - 0.050) / 0.012 = Z 1.97

▶ 값 0.12 → (0.12 - 0.050) / 0.012 = Z 5.74

값	Z-score	읽는 법
0.060	약 0.8	평소 흔들림 안쪽 — 평범한 값
0.074	약 2.0	가장자리 — 살펴볼 만함
0.120	약 5.7	표준편차 다섯 칸 초과 — 강한 이상 의심

개념

나눠셈이 하는 일이 특히 중요합니다. rms는 0.05 근처의 작은 숫자고 spectral_centroid는 1979 근처의 큰 숫자인데, Z-score로 바꾸면 둘 다 "몇 칸"이라는 같은 단위가 됩니다. 그래서 소리 세기와 톤을 나란히 놓고 "어느 쪽이 더 튀었나"를 비교할 수 있습니다. 교안이 이것을 단위 제거라고 부릅니다.

1-4. 부호는 방향, 절댓값은 크기

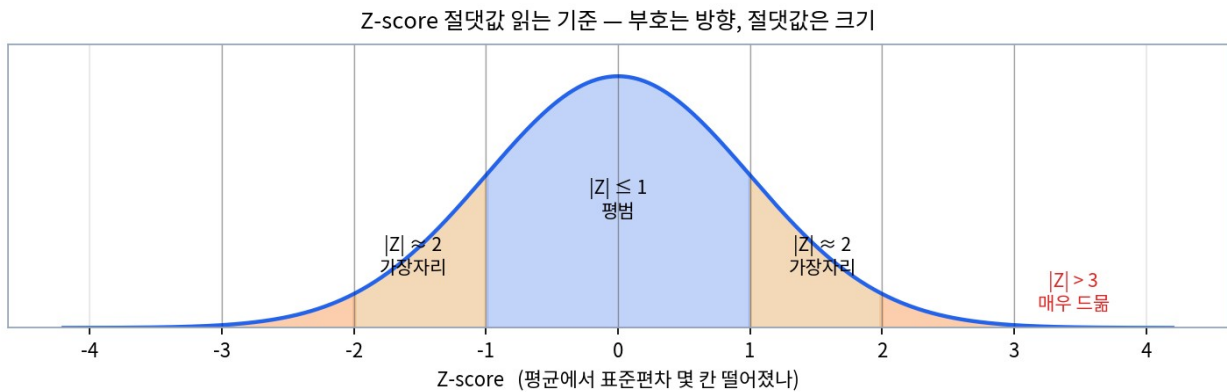
Z-score는 음수도 나옵니다. 부호와 절댓값을 각각 다르게 읽어야 합니다.

읽는 것	무엇을 말하나	예시
부호	평균보다 큰지 작은지 — 어느 방향으로 벗어났나	큰 양수 = 평소보다 세짐 · 큰 음수 = 평소보다 약해짐
절댓값	부호를 떼고 본 크기 — 얼마나 멀리 튀었나	+3이든 -3이든 똑같이 세 칸 떨어짐

개념

이상 여부는 절댓값으로 판단하고, 부호는 원인 분석을 위해 함께 기록합니다. 소리가 커진 것과 작아진 것은 둘 다 이상 신호일 수 있지만 원인이 다릅니다 — 베어링 마모나 부품 헐거움이면 소리가 커지고, 힘을 못 내는 고장이면 소리가 작아집니다. 그래서 판정에는 절댓값을, 보고에는 부호까지 씁니다.

절댓값 크기별로 어떻게 읽는지는 이렇게 정리됩니다. 다음 그림이 이 구간을 눈으로 보여 줍니다.



▲ Z-score 구간별 해석 — 종 모양 분포에서 $|Z| \leq 1$ 이 약 68%, $|Z| \leq 2$ 가 약 95%, $|Z| \leq 3$ 이 약 99.7%를 차지합니다

절댓값	판정	뜻
-1 ~ 1	평범	평소 범위 한가운데에 머무는 값
약 2	가장자리	꽤 가장자리에 있는 값 — 의심스러워 살펴볼 만함
3 초과	매우 드물	평소엔 거의 안 나오는 값 — 강한 이상 의심

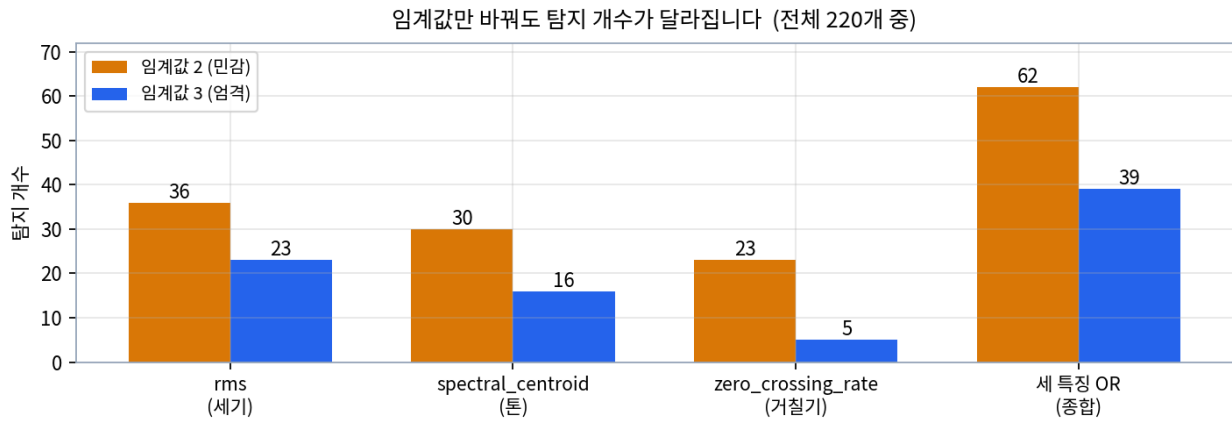
1-5. 임계값 — 통계가 아니라 우리가 정하는 선

Z-score를 다 구했다고 판단이 끝난 것은 아닙니다. 몇 점부터 이상으로 칠지를 정해야 비로소 판정이 완성됩니다. 그 기준선이 임계값입니다.

개념

임계값은 통계가 정해 주지 않습니다. 우리가 직접 돌리는 다이얼입니다. 교안의 비유가 정확합니다 — 화재경보기 민감도와 같습니다. 너무 낮으면 토스트만 구워도 울리고, 너무 높으면 진짜 불이 나야 울립니다. 어느 쪽이 나은지는 그 방이 어떤 방이냐에 달려 있습니다.

오늘 데이터에서 임계값 2와 3을 각각 적용하면 탐지 개수가 이렇게 달라집니다. 같은 데이터, 같은 Z-score인데 경계선만 옮긴 것입니다.



▲ 임계값 2(주황)와 3(파랑)의 탐지 개수 — 임계값을 낮추면 rms 기준 23개가 36개로, 종합 기준 39개가 62개로 늘어납니다

구분	임계값 2 (민감)	임계값 3 (엄격)
포함 범위	약 95% 안쪽	약 99.7% 안쪽
이상 비율	약 5% 바깥	약 0.3% 바깥
rms 탐지 수	36개	23개
세 특징 종합	62개	39개
성격	많이 잡음 — 놓침 적고 헛경보 많음	적게 잡음 — 헛경보 적고 놓침 많음

주의

2와 3은 관습적인 출발점일 뿐입니다. 객관식 두 개 중에 고르는 것이 아니라 자유롭게 돌리는 다이얼입니다. 교안이 제시한 실무 습관은 보통 3으로 시작해서 결과를 보며 2 쪽으로 조정하는 것입니다. 핵심 설비는 민감하게(2 쪽으로), 보조 설비는 엄격하게(3 쪽으로) 잡습니다.

1-6. 두 가지 실수 — 거짓 경보와 놓침

임계값을 정할 때 무엇을 저울질하는지가 이 절의 주제입니다. 이상탐지에서 저지를 수 있는 실수는 두 종류이고, 성격이 완전히 다릅니다.

실수		
무슨 일	정상인데 이상이라 잘못 알림	이상인데 정상이라 지나침
당장의 결과	정비사가 헛걸음	진짜 고장을 막지 못함
쌓이면	경보를 신뢰하지 않게 됨 — 시스템 전체 불신	큰 사고 · 설비 · 인명 피해

주의

거짓 경보가 무서운 이유는 비용 때문이 아니라 신뢰 때문입니다. 헛걸음 한 번의 비용은 크지 않지만, 그것이 반복되면 다음 경보도 무시하게 됩니다. 그러면 진짜 경보가 왔을 때도 아무도 안 움직입니다 — 즉 거짓 경보가 쌓이면 결국 농침으로 바뀝니다.

1-7. 시소의 법칙 — 둘을 동시에 줄일 수는 없습니다

그럼 임계값을 잘 맞춰서 두 실수를 모두 줄이면 되지 않을까요. **안 됩니다.** 이것이 이 단원에서 가장 중요한 실무 감각입니다.

임계값 조정	농침	거짓 경보
낮춤 (3 → 2)	줄어들	늘어남
높임 (2 → 3)	늘어남	줄어들

개념

한쪽을 좋게 하면 반드시 다른 쪽이 나빠지는 시소 관계입니다. 임계값은 두 실수 사이를 오가는 다이얼일 뿐이고, 양쪽을 한꺼번에 줄이는 마법은 없습니다. 두 지표를 동시에 올리려면 임계값이 아니라 더 나은 특징이나 더 나은 방법이 필요합니다 — 그래서 CH 5에서 IsolationForest로 넘어가는 것입니다.

그렇다면 다이얼을 어느 쪽으로 돌려야 할까요. 기준은 하나입니다 — 무엇이 더 치명적인가.

상황	판단	다이얼 방향
핵심 펌프 — 멈추면 라인 전체가 섬	헛걸음 몇 번이 대형 사고보다 쌉니다	낮은 임계값 (민감하게)
보조 설비 — 예비기 있고 복구 쉬움	거짓 경보 피로가 더 부담입니다	높은 임계값 (엄격하게)

연결

33일차의 재현율·정밀도 이야기가 여기서 다시 나옵니다. 농침을 줄이는 것이 재현율을 올리는 것이고, 거짓 경보를 줄이는 것이 정밀도를 올리는 것입니다. 그때 배운 "감당 가능한 경보 건수 상한을 먼저 정한다"는 순서가 오늘 임계값을 정하는 데도 그대로 적용됩니다.

주의

한 번 정하고 끝이 아닙니다. 경보 결과와 정비 결과를 보며 점진적으로 보정해야 합니다. 처음부터 완벽한 임계값을 찾으려 하기보다, 3에서 시작해 한두 달 운영해 보고 "너무 자주 올린다" 또는 "농침 게 있다"는 피드백으로 조정하는 편이 현실적입니다.

MIMI 데이터 이해

소리를 숫자 세 개로 요약한다는 것

2-1. 특징값이란 무엇인가

우리가 다루는 것은 소리 파일이 아니라 **특징값 CSV**입니다. 왜 그렇게 하는지부터 짚고 가겠습니다.

소리를 1초만 녹음해도 **수만 개의 숫자**가 나옵니다. 마이크가 초당 수만 번 공기의 진동을 재기 때문입니다. 그 수만 개를 그대로 비교하는 것은 불가능하니, **몇 개의 대표 숫자로 압축**합니다. 그 대표 숫자가 특징값입니다.

비유

교안의 비유가 좋습니다 — **사람을 소개할 때 사진과 영상을 전부 보여 주는 대신 키·몸무게·나이 같은 대표 숫자 몇 개로 요약**하는 것과 같습니다. 정보 일부는 사라지지만, **비교하기에는 훨씬 편합니다**.

이득	내용
가볍다	수만 개 숫자가 서너 개로 — 메모리·연산 부담이 사라짐
익숙하다	평균·표준편차·Z-score·그래프 — 지금까지 배운 도구를 그대로 씀
단서가 된다	정상·이상 차이가 특징값에 실제로 드러남

주의

요약 과정에서 정보 일부는 반드시 사라집니다. 그래서 어떤 특징을 뽑느냐가 분석의 성패를 좌우합니다. 예를 들어 소리의 세기만 뽑아 두면, 세기는 그대로인데 톤만 변하는 고장은 영원히 안 보입니다 — 오늘 CH 4의 핵심 이야기가 정확히 이것입니다.

2-2. 세 특징이 각각 무엇을 보나

오늘 데이터에는 특징이 세 개 있습니다. 이름이 낯설지만 **각각 소리의 다른 측면**을 봅니다.

특징	한국어	무엇을 재나	일상 비유
rms	세기	소리가 평균적으로 얼마나 센지	볼륨
spectral_centroid	톤	에너지가 높은 음 쪽인지 낮은 음 쪽인지	시소의 무게중심
zero_crossing_rate	거칠기	신호가 중심선을 얼마나 자주 지나는지	모래와 자갈의 질감 차이

rms — 소리의 세기

rms는 Root Mean Square의 줄임말입니다. 31일차 진동 이야기에서 나온 그 RMS와 같은 계산입니다 — **파형 전체를 대표하는 실효값**입니다. 직관적으로는 볼륨이라고 생각하시면 됩니다.

상태 변화	rms의 반응
베어링 마모 · 부품 헐거움	상승 — 소리가 세짐
힘을 못 내는 고장	하락 — 소리가 약해짐

개념

어느 쪽이든 상태 변화 신호입니다. 1-4절에서 "부호는 방향, 절댓값은 크기"라고 한 이유가 이것입니다 — Z-score가 +5든 -5든 둘 다 이상이고, 어느 쪽인지는 원인을 좁히는 데 씁니다.

spectral_centroid — 소리의 톤

이름이 어렵지만 뜻은 단순합니다. 소리 에너지의 무게중심이 어디에 있느냐입니다. 시소 위에 무게추를 올린다고 생각해 보십시오. 오른쪽(높은 음)에 추가 많으면 무게중심도 오른쪽으로 갑니다.

값	뜻	들리는 느낌
클 때	높은 주파수 에너지가 많음	밝고 날카로운 톤
작을 때	낮은 주파수 에너지가 많음	둔하고 묵직한 톤

개념

부품이 마모되거나 금이 가면 날카로운 고주파 진동이 생깁니다. 그러면 무게중심이 위로 끌려 올라갑니다. 그래서 무게중심 상승은 "어딘가 거칠어지기 시작했다"는 초기 신호로 읽힙니다. 오늘 데이터에서 정상 평균은 1979인데, 일부 이상은 2700~3000까지 올라갑니다.

zero_crossing_rate — 소리의 거칠기

줄여서 ZCR이라고 부릅니다. 신호가 0(중심선)을 얼마나 자주 지나는지를 센 값입니다. 파형이 잔잔하면 중심선을 드물게 지나고, 잡음이 섞여 부들부들 떨리면 자주 지납니다.

비유

매끄러운 모래와 거친 자갈을 손으로 만져 구분하는 것과 같습니다. 소리의 질감을 숫자 하나로 옮긴 셈입니다. 마모나 윤활 부족으로 거친 마찰음이 생기면 값이 커집니다.

주의

세 특징은 서로를 보완합니다. 한 특징이 놓친 변화를 다른 특징이 잡습니다. rms로 안 잡히고 톤으로도 안 잡히는 미묘한 거칠기 변화를 ZCR이 잡아 주기도 합니다. 그래서 교안이 "항상 셋을 함께 살피는 습관"을 강조합니다.

2-3. label — 정답이 붙어 있는 특별한 데이터

실습 데이터에는 label 컬럼이 있습니다. 정상은 0, 이상은 1입니다. 그런데 0-3절에서 "이상탐지는 비지도학습"이라고 했는데 정답이 있다니 모순처럼 들립니다.

시점	label을 쓰나	왜
이상을 찾을 때	안 씁니다	현장에는 정답이 없으므로 그 상황을 똑같이 재현
결과를 채점할 때	씁니다	우리가 얼마나 잘 찾았는지 확인하려면 정답이 필요

비유

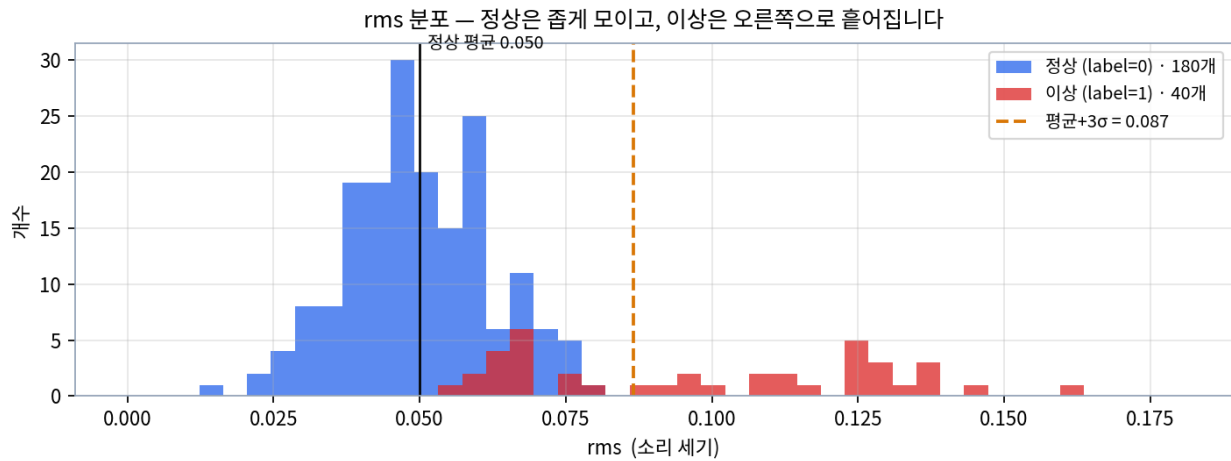
시험은 답안지를 덮고 풀고, 채점할 때만 답안지를 펼칩니다. label도 똑같습니다. Z-score를 계산할 때는 label 컬럼을 쓰지만 정상 기준선을 고르는 용도로만 쓰고, 판정 자체는 특징값만 보고 합니다. 그리고 판정이 끝난 뒤 채점할 때 다시 꺼냅니다.

주의

이것은 연구·교육용 특례입니다. 실제 현장 설비에는 이런 정답이 보통 없습니다. 그래서 33일차에서 "채점은 학습 데이터에서만 가능한 연습"이라고 한 것이고, 오늘 26_mimii_new.csv에 label이 없는 이유도 그 현장 상황을 그대로 보여 주기 위해서입니다.

2-4. 정상과 이상의 분포 차이

이상탐지가 가능한 근본 이유는 결국 하나입니다 — 정상과 이상의 분포가 다르기 때문입니다. 말로 설명하기보다 그림이 훨씬 빠릅니다.



▲ 실습 데이터의 rms 히스토그램 — 정상(파랑)은 0.050 부근에 봉긋한 산을 이루고, 이상(빨강)은 그 오른쪽으로 넓게 흩어집니다

구분	모양	왜 그런가
정상	평균 중심으로 좁게 모여 산 모양	평소 비슷한 값이 자주 나오니 가운데가 봉긋해짐
이상	그 산에서 멀리 떨어져 번두리에 흩어짐	종류가 다양하고 수가 적어 드문드문

개념

그림에서 두 가지를 보시면 됩니다. ① 파란 산이 0.050 근처에 좁게 몰려 있고, ② 빨간 점들이 0.09부터 0.16까지 넓게 퍼져 있습니다. 주황 점선(평균+3σ = 0.087)이 그 둘을 가르는 선인데, **그 선 오른쪽은 거의 빨강입니다.** 이것이 Z-score가 잘 작동하는 이유입니다.

주의

그런데 그림을 자세히 보면 0.05~0.08 구간에 빨강이 파랑과 겹쳐 있습니다. 이 겹친 부분이 나중에 문제가 됩니다 — rms만 보면 이 이상들은 정상 속에 숨어서 안 잡힙니다. CH 4-2절에서 이 숨은 이상 17개를 정면으로 다룹니다.

2-5. 분석 환경 — 네 도구의 역할

코드 맨 위에서 한 번만 불러오면 계속 쓰는 도구 네 개입니다. **전부 이미 앞 단원에서 만나 본 것들**이라 새로 배울 것은 없습니다.

거의 모든 데이터 분석 코드의 첫 네 줄

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

별명	정식 이름	역할	비유
pd	pandas	CSV 불러오기, 표 다루기	데이터 그릇

별명	정식 이름	역할	비유
np	numpy	절댓값 등 숫자 계산	계산 엔진
plt	matplotlib	기본 그래프 그리기	그림 붓
sns	seaborn	보기 좋은 그래프 자동 옷	붓 도우미

개념

별명(`pd·np·plt·sns`)은 거의 전 세계 공통 관습입니다. 인터넷에서 남의 코드를 봐도 이 네 글자는 똑같이 쓰입니다. 그래서 `pd.read_csv(...)`를 보면 "아 pandas구나" 하고 바로 알 수 있습니다. **역할만 기억해 두면** 낯선 코드도 절반은 읽힙니다.

주의

오늘 CH 5부터 다섯 번째 도구인 `sklearn(scikit-learn)`이 추가됩니다. 앞의 넷과 성격이 다릅니다 — 앞의 넷은 "데이터를 다루고 그리는" 도구인데, `sklearn`은 "모델을 만들고 학습시키는" 도구입니다. CH 5-6절에서 처음부터 설명합니다.

CHAPTER 3

Z-score 적용과 해석

기준선 → 계산 → 판정, 그리고 정비 문장으로

3-1. 세 단계 흐름

CH 1에서 배운 개념이 실제 코드에서는 **세 단계**로 압축됩니다. 이 흐름만 머리에 있으면 코드가 술술 읽힙니다.



단계	하는 일	만들어지는 것
STEP 1	정상만 골라 평균·표준편차 구하기	mu, sd 확보
STEP 2	전체 데이터에 (값-평균)/표준편차 적용	z_rms 같은 점수 컬럼
STEP 3	절댓값이 임계값을 넘으면 이상으로 표시	is_anomaly 같은 판정 컬럼

개념

특징이 세 개면 이 흐름을 세 번 반복합니다. rms엔 rms 기준선, 톤엔 톤 기준선을 각각 씁니다. 흐름은 똑같고 대상만 갈아 끼우는 방식이라 부담이 적습니다.

3-2. 실습 3 — 정상 기준선 세우기

메서드 `.mean()` / `.std()` 평균과 표준편차 계산

정의 `.mean()`은 평균을, `.std()`는 표준편차를 돌려주는 pandas 메서드입니다. 21일차에 배운 그것과 같습니다.

형태 `시리즈.mean()` · `시리즈.std()`

```
# 정상(label=0)만 골라내기 — 1-2절 원칙
normal = df[df["label"] == 0]
print("정상 개수:", len(normal))
print()

# 세 특징의 기준선을 한 번에
for c in ["rms", "spectral_centroid", "zero_crossing_rate"]:
    print(f"{c:20} 평균 {normal[c].mean():10.4f}    표준편차 {normal[c].std():9.4f}")
```

▶ 정상 개수: 180

▶

▶ rms 평균 0.0500 표준편차 0.0122

▶ spectral_centroid 평균 1979.0989 표준편차 239.1756

▶ zero_crossing_rate 평균 0.0910 표준편차 0.0189

왜 쓰나 세 특징의 숫자 크기가 완전히 다릅니다. rms는 0.05, 톤은 1979, 거칠기는 0.091입니다. 이 상태로는 "어느 쪽이 더 튀었나"를 비교할 수 없는데, Z-score로 바꾸면 셋 다 "몇 칸"이라는 같은 단위가 됩니다.

응용 `df[df["label"] == 0]`이라는 조건 인덱싱이 이 실습의 핵심입니다. 만약 이 조건을 빼면 어떻게 되는지 나란히 확인해 보면 원칙의 이유가 분명해집니다.

```
for c in ["rms"]:
    a = df[df["label"] == 0][c]    # 정상만
    b = df[c]                      # 전체 (이상 섞임)
    print(f"정상만 평균 {a.mean():.4f}    표준편차 {a.std():.4f}")
    print(f"전체    평균 {b.mean():.4f}    표준편차 {b.std():.4f}")
```

▶ 정상만 평균 0.0500 표준편차 0.0122

▶ 전체 평균 0.0589 표준편차 0.0254

개념

표준편차가 0.0122에서 0.0254로 두 배 넘게 부풀었습니다. 평균도 0.050에서 0.059로 끌려갔습니다. 이 오염된 기준선을 쓰면 이상이 정상 범위 안에 숨습니다 — CH 4-4절에서 이 효과를 숫자로 확인합니다.

자주 하는 실수

`.std()`는 pandas와 numpy가 기본값이 다릅니다. pandas의 `.std()`는 **표본 표준편차**($n-1$ 로 나눔)이고, numpy의 `.std()`는 **모집단 표준편차**(n 으로 나눔)입니다. 오늘 데이터는 180개라 차이가 미미하지만, 데이터가 적을 때는 결과가 달라질 수 있습니다.

3-3. 실습 4 — Z-score 계산과 가장 튼 값 찾기

패턴 Z-score 컬럼 만들기 전체 데이터에 식을 한 번에 적용

정의 pandas는 시리즈 전체에 사칙연산을 한 번에 적용합니다. 반복문 없이 한 줄로 220행 전부가 계산됩니다.

형태 `df["z_rms"] = (df["rms"] - mu) / sd`

```
normal = df[df["label"] == 0]
mu, sd = normal["rms"].mean(), normal["rms"].std()

# 220행 전체에 한 줄로 적용
df["z_rms"] = (df["rms"] - mu) / sd

print(df[["rms", "z_rms", "label"]].head(5).round(3).to_string())
```

	rms	z_rms	label
0	0.066	1.305	0
1	0.039	-0.874	0
2	0.044	-0.465	0
3	0.046	-0.342	0
4	0.070	1.624	0

왜 쓰나 반복문이 필요 없다는 점이 pandas의 힘입니다. `df["rms"] - mu`가 220개 값에서 각각 mu를 빼고, `/ sd`가 220개를 각각 나눕니다. 이것을 **벡터 연산**이라고 부르는데, 반복문보다 훨씬 빠릅니다.

응용 가장 튼 값을 찾으려면 절댓값 기준으로 정렬합니다. `.abs()`로 부호를 떼고 `.sort_values()`로 큰 순서로 놓습니다.

```
df["z_rms_abs"] = df["z_rms"].abs()
top = df.sort_values("z_rms_abs", ascending=False).head(5)
print(top[["rms", "z_rms", "label"]].round(3).to_string())
```

	rms	z_rms	label
70	0.160	9.006	1
108	0.147	7.924	1
192	0.138	7.212	1
164	0.136	7.072	1
26	0.135	6.990	1

개념

가장 튼 값은 70번 샘플이고 Z가 9.006입니다. 평균에서 표준편차 아홉 칸이나 떨어졌다는 뜻입니다. 1-4절 기준표에서 "3 초과는 매우 드물"이었으니, 9는 **극단적으로 드문 값**입니다. 그리고 상위 다섯 개가 전부 label=1(실제 이상)입니다.

설비 적용

실무에서는 이 정렬 결과가 곧 **점검 우선순위 목록**이 됩니다. "이상이 23건 있습니다"보다 "**1순위는 70번, Z가 9입니다**"가 현장에서 훨씬 잘 받아들여집니다.

3-4. 실습 5 — 임계값 적용과 2·3 비교

메서드 `.abs()` 절댓값 — 부호를 떼기

정의 양수는 그대로 두고 음수는 부호만 떼어 크기만 남깁니다. 1-4절에서 "이상 여부는 절댓값으로 판단"한다고 했으니 판정 전에 반드시 거칩니다.

형태 `시리즈.abs()`

```
# 절댓값이 3을 넘으면 이상으로 표시
df["is_anomaly"] = (df["z_rms"].abs() > 3).astype(int)

print("임계값 3 탐지:", int(df["is_anomaly"].sum()), "건")
print("임계값 2 탐지:", int((df["z_rms"].abs() > 2).sum()), "건")
```

▶ 임계값 3 탐지: 23 건

▶ 임계값 2 탐지: 36 건

왜 쓰나 임계값을 3에서 2로 낮추니 23건이 36건이 되었습니다. 13건이 더 잡힌 것인데, 이 13건이 진짜 이상인지 헛정보인지는 label로 채점해 봐야 합니다.

응용 `.astype(int)`이 하는 일도 짚어 두겠습니다. > 비교의 결과는 True/False인데, 여기에 `.astype(int)`를 붙이면 1/0이 됩니다. 그러면 `.sum()`으로 개수를 셀 수 있습니다.

```
for th in [2, 3]:
    d = (df["z_rms"].abs() > th)
    tp = int((d & (df.label == 1)).sum()) # 진짜 이상을 잡음
    fp = int((d & (df.label == 0)).sum()) # 오탐
    fn = int((~d & (df.label == 1)).sum()) # 놓침
    print(f"임계 {th} 탐지 {int(d.sum()):2}건 · 진짜 {tp:2} 오탐 {fp:2} 놓침 {fn:2}")
```

▶ 임계 2 탐지 36건 · 진짜 26 오탐 10 놓침 14

▶ 임계 3 탐지 23건 · 진짜 23 오탐 0 놓침 17

개념

1-7절의 시소가 숫자로 나타났습니다. 임계값 3에서는 오탐이 0건으로 완벽하지만 17건을 놓쳤고, 2로 낮추니 놓침이 14건으로 줄어든 대신 오탐이 10건 생겼습니다. 3건을 더 건지려고 헛경보 10건을 받은 셈입니다. **어느 쪽이 나은지는 이 설비가 얼마나 중요하냐에 달려 있습니다.**

자주 하는 실수

&와 ~는 pandas에서 **and**와 **not**을 뜻하는 기호입니다. 파이썬의 **and**·**not**을 쓰면 시리즈끼리는 오류가 납니다. 그리고 **괄호를 반드시 쳐야** 합니다 — `(d) & (df.label==1)` 처럼요. 우선순위 때문에 괄호를 빼면 엉뚱한 결과가 나옵니다.

3-5. 실습 6 — 세 특징 종합 판정

이제 특징 하나가 아니라 **셋을 모두** 봅니다. 판정 규칙은 단순합니다 — **하나라도 넘으면 이상**입니다.

패턴 세 특징 OR 결합 하나라도 넘으면 이상으로 종합

정의 세 특징의 Z-score 절댓값을 한꺼번에 임계값과 비교한 뒤, `.any(axis=1)`로 "행마다 하나라도 True가 있나"를 봅니다.

형태 `df[["z_a", "z_b", "z_c"]].abs().gt(3).any(axis=1)`

```
F = ["rms", "spectral_centroid", "zero_crossing_rate"]
normal = df[df["label"] == 0]

# 세 특징마다 기준선을 따로 세워 Z-score 계산
for c in F:
    df["z_" + c] = (df[c] - normal[c].mean()) / normal[c].std()

# 하나라도 절댓값 3을 넘으면 이상
zcols = ["z_" + c for c in F]
df["z_anom"] = df[zcols].abs().gt(3).any(axis=1).astype(int)

print("rms 단독:", int((df["z_rms"].abs() > 3).sum()), "건")
print("세 특징 종합:", int(df["z_anom"].sum()), "건")
```

▶ rms 단독: 23 건

▶ 세 특징 종합: 39 건

왜 쓰나 23건에서 39건으로 16건이 더 잡혔습니다. rms만 봤을 때는 안 보이던 이상이, 톤이나 거칠기를 함께 보니 드러난 것입니다. 특징을 늘리는 것만으로 탐지력이 크게 올라갔습니다.

응용 `.any(axis=1)`이 무엇을 하는지 풀어 보겠습니다. `axis=1`은 **가로 방향(행 단위)**으로 보라는 뜻입니다. 각 행에서 세 값 중 하나라도 True면 그 행은 True가 됩니다.

```
sample = df[zcols].abs().gt(3).head(3)
print(sample.to_string())
print()
print("행별 any(axis=1):")
print(sample.any(axis=1).to_string())
```

```
▶   z_rms  z_spectral_centroid  z_zero_crossing_rate
▶ 0  False                    False                  False
▶ 1  False                    False                  False
▶ 2  False                    False                  False
▶
▶ 행별 any(axis=1):
▶ 0  False
▶ 1  False
▶ 2  False
```

개념

`axis=0`은 세로(컬럼 단위), `axis=1`은 가로(행 단위)입니다. 헷갈릴 때는 **"어느 방향으로 접느냐"**로 기억하시면 됩니다 — `axis=1`은 가로로 접어서 행마다 값 하나를 남깁니다.

설비 적용

이 OR 결합이 CH 4의 한계 이야기로 이어집니다. 각 특징을 따로 보고 나중에 결과만 합친 것이라, **특징들 사이의 관계는 전혀 보지 못합니다.** 그것이 Z-score의 구조적 한계이고, IsolationForest가 필요한 이유입니다.

세 방법의 채점 결과를 나란히 놓으면 차이가 분명합니다.

방법	탐지	진짜 이상	오탐	놓침
rms 단독 (임계 3)	23건	23	0	17
세 특징 OR (임계 3)	39건	37	2	3

개념

놓침이 17건에서 3건으로 줄었습니다. 오탐은 0건에서 2건으로 늘었지만 그 대가로 14건을 더 건졌습니다. 1-7 절의 시소 관계지만 **이번에는 임계값을 옮긴 것이 아니라 보는 특징을 늘린 것**이라, 두 지표가 거의 함께 좋아졌습니다. 33일차의 표현을 빌리면 **곡선 자체를 위로 밀어 올린 것**입니다.

3-6. 실습 7 — 결과를 정비 언어로 옮기기

탐지 결과를 숫자로만 던지면 현장에서 아무 일도 일어나지 않습니다. **사람이 읽고 움직일 수 있는 문장**으로 바뀌어야 합니다. 좋은 번역에는 네 가지가 들어갑니다.

요소	물는 것	예시
대상	어느 샘플인가	12번 샘플
특징	어느 측면인가	소리 세기
정도	얼마나 벗어났나	표준편차 5.7배
방향	어느 방향인가	위로 벗어남

"12번 샘플은 소리 세기가 평소 표준편차 5.7배 위로 벗어났습니다"

정비 문장 자동 생성

```
# 이상으로 표시된 것 중 Z 절댓값 상위 3개를 문장으로
anom = df[df["z_anom"] == 1].copy()
anom["z_max"] = anom[zcols].abs().max(axis=1)

name = {"z_rms": "소리 세기", "z_spectral_centroid": "톤",
        "z_zero_crossing_rate": "거칠기"}

for i, row in anom.nlargest(3, "z_max").iterrows():
    col = anom.loc[i, zcols].abs().idxmax() # 가장 많이 튜 특징
    z = df.loc[i, col]
    dirn = "위로" if z > 0 else "아래로"
    print(f"{i}번 샘플은 {name[col]}가 평소 표준편차 "
          f"{abs(z):.1f}배 {dirn} 벗어났습니다 — 점검을 권합니다")
```

- ▶ 70번 샘플은 소리 세기가 평소 표준편차 9.0배 위로 벗어났습니다 — 점검을 권합니다
- ▶ 108번 샘플은 소리 세기가 평소 표준편차 7.9배 위로 벗어났습니다 — 점검을 권합니다
- ▶ 192번 샘플은 소리 세기가 평소 표준편차 7.2배 위로 벗어났습니다 — 점검을 권합니다

× 나쁜 표현	✓ 좋은 표현	왜
"고장입니다"	"점검을 권합니다"	이상탐지가 잡은 것은 고장이 아니라 "평소와 다름"
무작위 나열	Z-score 큰 순서	현장은 무엇부터 봐야 할지를 알아야 움직임
과장 · 축소	숫자가 말하는 만큼만	과장하면 신뢰를 잃고, 축소하면 놓칩니다
숫자만 던지기	대상 · 특징 · 정도 · 방향	네 가지가 다 있어야 행동이 나옵니다

개념

"고장입니다"라고 쓰지 않는 이유가 중요합니다. Z-score가 크다는 것은 **평소와 다르다**는 뜻이지 고장이라는 뜻이 아닙니다. 운전 조건이 바뀌었을 수도 있고, 센서 문제일 수도 있습니다 — 31일차에 배운 그 판별 세 질문이 여기서도 필요합니다. **정직하고 행동 가능한 보고**의 핵심은 아는 만큼만 말하는 것입니다.

3-7. 실습 8 — Z-score가 놓친 이상 확인

마지막 실습은 **rms 하나로는 무엇을 놓쳤는지** 확인하는 것입니다. 이 결과가 CH 4의 출발점이 됩니다.

놓친 17건의 정체

```
# 임계값을 안 넘었는데 실제로는 이상인 샘플
det = df["z_rms"].abs() > 3
missed = (~det) & (df["label"] == 1)
```

```
print("rms 단독으로 놓친 실제 이상:", int(missed.sum()), "건")
print()
print(df[missed][["rms", "z_rms", "z_spectral_centroid",
                  "z_zero_crossing_rate"]].round(2).head(8).to_string())
```

▶ rms 단독으로 놓친 실제 이상: 17 건

```
▶
▶      rms  z_rms  z_spectral_centroid  z_zero_crossing_rate
▶ 30  0.06   1.23                4.06                2.42
▶ 35  0.07   1.40                3.83                2.91
▶ 45  0.06   0.68                3.58                3.27
▶ 63  0.07   1.33                3.36                2.73
▶ 74  0.07   1.57                2.47                3.56
▶ 75  0.07   1.60                3.38                2.57
▶ 81  0.08   2.11                4.50                2.86
▶ 83  0.07   1.24                3.53                2.02
```

개념

패턴이 보입니다. 놓친 샘플들은 z_rms가 0.6~2.1로 평범한데, z_spectral_centroid(톤)과 z_zero_crossing_rate(거칠기)가 3을 넘습니다. 즉 세기는 보통인데 톤이 높고 거칠어진 이상입니다. **2-2 절에서 예고한 "세기는 그대로인데 톤만 변하는 고장"이 실제로 데이터에 들어 있었습니다.**

주의

17건 중 한 건(137번)은 성격이 다릅니다. z_rms가 2.976으로 임계값 3에 아주 근소하게 못 미쳤고, 톤·거칠기도 정상 범위입니다. 이건 "다변량 이상"이 아니라 **경계선에 걸린 가까운 케이스**입니다. 임계값을 2.9로만 낮춰어도 잡혔을 샘플이라, 1-7절의 시소가 실제로 어떻게 작동하는지를 보여 주는 예입니다.

CHAPTER 4

Z-score의 한계

왜 다음 도구가 필요한가

4-1. 한계 1 — 특징을 하나씩만 봅니다

3-5절에서 세 특징을 종합했는데, 그것은 각 특징을 따로 판정한 뒤 결과만 나중에 합친 것입니다. 이것을 **OR 결합**이라고 부릅니다.



개념

각 특징을 하나씩 따로 보는 1차원 판단입니다. rms는 rms만, 톤은 톤만 봅니다. 그러니 특징들 사이의 관계는 전혀 보지 못합니다. 각각은 정상 범위 안인데 조합이 비정상인 경우에 원리적으로 무력합니다.

4-2. 조합 이상 — 키와 몸무게로 이해하기

말로만 하면 추상적이니 일상 예로 옮겨 보겠습니다.

항목	값	따로 보면	함께 보면
키	175 cm	정상	
몸무게	50 kg	정상	
조합	175cm · 50kg	—	꽤 마름 — 특이한 조합

키 175cm는 흔하고 몸무게 50kg도 흔합니다. 각각 따로 보면 아무 문제가 없습니다. 그런데 둘을 함께 보면 꽤 마른 체형이라는 것이 드러납니다. Z-score는 키를 따로 재고 몸무게를 따로 재기 때문에 이 조합을 영원히 못 잡습니다.

개념

설비도 똑같습니다. 정상 기계는 특징들이 함께 움직입니다 — 세기가 보통이면 톤도 거칠기도 보통입니다. 그런데 고장이 나면 그 관계가 어긋납니다 — 세기는 그대로인데 톤만 높아지는 식입니다. 이런 것을 **다변량 이상**이라고 부릅니다.

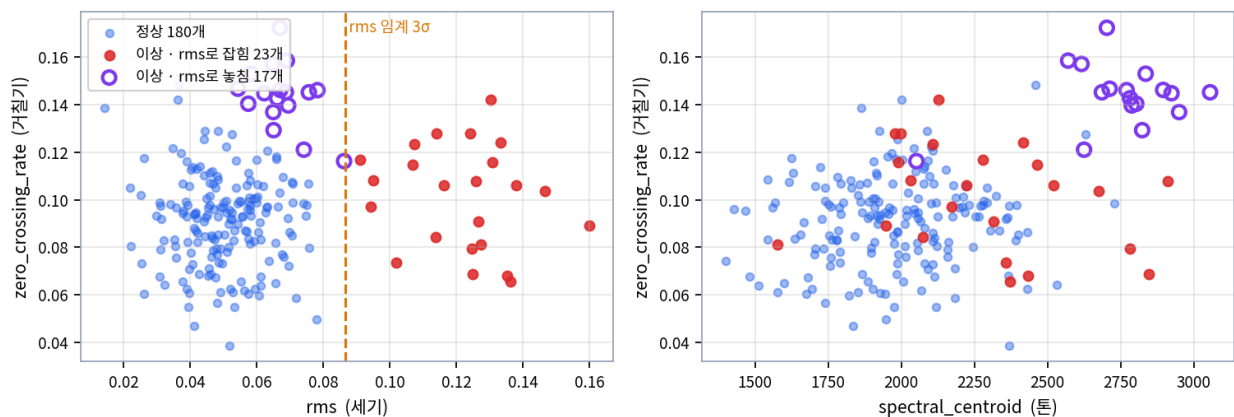
4-3. 실습 데이터의 두 종류 이상

3-7절에서 rms 단독으로 17건을 놓쳤다고 했습니다. 그 17건을 성격별로 나누면 이렇습니다.

종류	특징	개수	Z-score로
명백한 이상	rms가 크게 튼 (z 5 초과)	23건	쉽게 잡힘 — 1차원으로 충분
숨은 이상	rms 평범(z 0.4~2.1)인데 톤·거칠기만 높음	16건	rms만 보면 통째로 놓침
경계선 케이스	rms z가 2.976 — 임계값 3에 근소하게 못 미침	1건	임계값을 조금만 낮췄으면 잡힘

이 차이를 **그림으로 보면 훨씬 분명합니다**. 왼쪽은 rms를 가로축에 둔 그림이고, 오른쪽은 rms를 아예 뺀 그림입니다.

한 축에서는 정상에 숨고, 두 축 평면에서는 따로 떨어져 보입니다



▲ 보라색 빈 원이 rms로 놓친 이상 17개 — 왼쪽 그림에서는 파란 정상 무리에 파묻혀 있지만, 오른쪽 그림(톤 × 거칠기)에서는 오른쪽 위에 따로 무리를 이룹니다

개념

왼쪽 그림을 보십시오. 보라 원들이 가로축 0.05~0.08 구간, 즉 파란 정상 점들과 완전히 섞인 자리에 있습니다. rms 임계선(주황 점선) 왼쪽이니 당연히 안 잡힙니다. 그런데 세로축(거칠기)으로는 위쪽에 몰려 있습니다 — 한 축에서는 숨었지만 다른 축에서는 이미 드러나 있었던 것입니다.

개념

오른쪽 그림이 결정적입니다. rms를 아예 빼고 톤과 거칠기만 놓으니, 보라 원들이 오른쪽 위에 자기들끼리 뚜렷한 무리를 이룹니다. 정상 무리와 확실히 떨어져 있습니다. 즉 이 이상들은 처음부터 보이는 곳에 있었고, 우리가 잘못된 축으로 보고 있었을 뿐입니다.

주의

교안이 이것을 "그림자"에 비유합니다. 3차원 물체를 한 방향으로 조명을 비춰 벽에 그림자를 드리우면, 어떤 방향에서는 두 물체가 겹쳐 보입니다. **rms 한 축으로 그림자를 드리우면 이상이 정상에 겹쳐 숨지만, 평면에서 보면 분명히 따로 있습니다.**

4-4. 한계 2 — 정규분포를 가정합니다

1-5절에서 "임계값 2가 약 95%, 3이 약 99.7%"라고 했습니다. 이 숫자에는 숨은 전제가 하나 있습니다 — 데이터가 **종 모양(정규분포)**이라는 가정입니다.

구분	내용	결과
가정	평균 중심으로 좌우 대칭인 종 모양	이 위에서만 95%·99.7% 비율이 성립
현실	한쪽으로 길게 치우치거나 봉우리가 둘이기도 함	설비 데이터는 울퉁불퉁한 경우가 많음
그래서	치우친 데이터에서는 퍼센트 해석이 어긋남	공식이 약속한 비율과 실제 비율이 다름

오늘 데이터로 직접 확인해 보겠습니다. **정말 임계값 3을 넘는 값이 0.3%뿐인지** 세어 보면 됩니다.

정상 구간만 놓고 실제 비율 확인

```
normal = df[df["label"] == 0]
```

```
for c in ["rms", "spectral_centroid", "zero_crossing_rate"]:  
    z = (normal[c] - normal[c].mean()) / normal[c].std()  
    over2 = (z.abs() > 2).sum() / len(z) * 100  
    over3 = (z.abs() > 3).sum() / len(z) * 100  
    print(f"{c:20} |Z|>2 {over2:5.2f}% (이론 5%)    "  
          f"|Z|>3 {over3:5.2f}% (이론 0.3%)")
```

```
▶ rms                |Z|>2  5.56% (이론 5%)    |Z|>3  0.00% (이론 0.3%)  
▶ spectral_centroid  |Z|>2  4.44% (이론 5%)    |Z|>3  0.56% (이론 0.3%)  
▶ zero_crossing_rate |Z|>2  3.89% (이론 5%)    |Z|>3  0.56% (이론 0.3%)
```

개념

이론값과 일추 비슷하지만 정확히 같지는 않습니다. $|Z|>2$ 는 이론이 5%인데 실제로는 5.56% · 4.44% · 3.89%로 특징마다 다릅니다. $|Z|>3$ 은 이론이 0.3%인데 톤과 거칠기에서 0.56%가 나왔습니다 — 180개 중 0.3%면 **0.54개**이니, 한 개만 나와도 0.56%가 되어 이론값의 두 배로 보입니다. **데이터가 적으면 퍼센트 해석 자체가 흔들립니다.**

주의

Z-score가 쓸모없다는 뜻이 아니라, 퍼센트 해석을 공식처럼 맹신하지 말라는 뜻입니다. 대응 방법은 단순합니다 — 임계값을 넘는 값을 직접 세어 실제 비율을 확인하면 됩니다. 그리고 그전에 분포를 먼저 그려 보는 습관이 그래서 중요합니다. 2-4절에서 히스토그램을 먼저 본 이유가 이것입니다.

4-5. 한계 3 — 평균과 표준편차가 흔들립니다

세 번째가 가장 치명적입니다. 평균과 표준편차는 극단값 몇 개에 쉽게 끌려갑니다.



3-2절 응용에서 이미 확인했지만, 여기서 탐지력이 실제로 얼마나 떨어지는지까지 계산해 보겠습니다.

기준선 오염의 대가

기준선을 두 가지로 만들어 비교

```
clean_mu = df[df.label==0]["rms"].mean() # 정상만 (올바름)
clean_sd = df[df.label==0]["rms"].std()
dirty_mu = df["rms"].mean() # 전체 (오염)
dirty_sd = df["rms"].std()
```

val = 0.120 # 명백한 이상 하나

```
print(f"정상만 기준: Z = {(val-clean_mu)/clean_sd:.2f}")
print(f"오염된 기준: Z = {(val-dirty_mu)/dirty_sd:.2f}")
print()
```

```
for name, mu_, sd_ in [("정상만", clean_mu, clean_sd), ("오염", dirty_mu, dirty_sd)]:
    z = ((df["rms"] - mu_) / sd_).abs()
    d = z > 3
    print(f"{name:6} 기준선 → 탐지 {int(d.sum()):2}건 · "
          f"진짜 이상 {int((d & (df.label==1)).sum()):2}건")
```

▶ 정상만 기준: Z = 5.74

▶ 오염된 기준: Z = 2.41

▶

▶ 정상만 기준선 → 탐지 23건 · 진짜 이상 23건

▶ 오염 기준선 → 탐지 5건 · 진짜 이상 5건

항목		
rms 평균	0.0500	0.0589

항목		
rms 표준편차	0.0122	0.0254 — 두 배 넘게 부족
값 0.120의 Z	5.74 — 명백한 이상	2.41 — 임계값 3도 못 넘음
임계 3 탐지 수	23건	5건 — 4분의 1도 못 잡음

주의

탐지력이 23건에서 5건으로 무너졌습니다. 명백한 이상(rms 0.120)조차 Z가 5.74에서 2.41로 떨어져 임계값 3을 못 넘습니다. 코드는 한 글자만 달랐는데 결과가 이렇습니다. 1-2절에서 "기준선은 정상 데이터로만 만든다"를 원칙으로 못 박은 이유가 이 숫자에 있습니다. 그리고 이것이 **구조적 약점**이라는 점이 중요합니다 — 임계값을 아무리 잘 조정해도 기준선이 오염되면 방법이 없습니다.

개념

이 한계가 다음 도구를 찾게 만드는 동기입니다. 현장에서는 "이 기간은 확실히 정상"이라고 자신 있게 말하기 어려울 때가 많습니다. 그럴 때 **평균·표준편차에 의존하지 않는 방법**이 필요하고, 그것이 CH 5의 IsolationForest입니다 — 오염에 훨씬 너그럽습니다.

4-6. 세 한계 정리 — 그리고 다음 장으로

한계	내용	어떻게 대응하나
① 한 특징만 본다	각각은 정상인데 조합이 비정상인 이상을 원리적으로 놓침	여러 특징을 동시에 보는 방법 → CH 5
② 정규분포 가정	치우친 데이터에서 퍼센트 해석이 어긋남	분포를 먼저 그리고, 실제 비율을 직접 세기
③ 평균이 흔들린다	이상이 섞이면 기준선이 오염되어 탐지력 붕괴	평균에 의존하지 않는 방법 → CH 5

개념

세 한계 중 ①과 ③을 동시에 해결하는 방법이 IsolationForest입니다. 여러 특징을 한꺼번에 보고(①), 평균·표준편차를 아예 쓰지 않습니다(③). 다만 ②는 그대로 남습니다 — 어떤 방법도 데이터 자체의 모호함은 없애지 못합니다.

주의

그렇다고 **Z-score**가 열등한 도구는 아닙니다. 오늘 실습에서 "세 특징 OR" 방식은 39건 탐지에 진짜 이상 37건, 오탐 2건이라는 **아주 좋은 성적**을 냈습니다. CH 7-5절에서 IsolationForest와 나란히 놓고 비교하는데, 결과가 의외입니다.

4-7. 한 장으로 정리하는 Z-score

CH 1~4를 통틀어 **Z-score**를 쓸 때 반드시 확인할 것을 순서대로 묶으면 이렇게 됩니다. 새 데이터를 만날 때마다 이 순서로 훑으시면 됩니다.

순서	확인할 것	안 하면 벌어지는 일
①	분포를 먼저 그려 본다 (히스토그램)	치우친 데이터인지 모르고 퍼센트를 맹신함
②	기준선을 정상만으로 계산한다	탐지력이 23건에서 5건으로 붕괴 (4-5절)
③	특징을 여러 개 함께 본다	한 특징만 보면 17건을 통째로 놓침 (4-3절)
④	임계값을 넘는 값을 직접 세어 본다	이론 비율과 실제 비율이 다를 수 있음 (4-4절)
⑤	부호를 함께 기록한다	원인 방향(세짐/약해짐)을 잃음 (1-4절)
⑥	두 특징 산점도로 눈으로 확인한다	숨은 이상이 있는지 알 방법이 없음

개념

②번이 가장 중요하고 가장 자주 틀립니다. 코드로는 `df[df.label==0]` 한 조각이지만, 현장에서는 **"어느 기간을 정상으로 볼 것인가"**라는 판단이 필요합니다. 33일차 라벨 설계에서 정비 이력으로 정상 구간을 정했던 것이 바로 이 준비 작업이었습니다.

주의

①과 ⑥이 모두 **"그림을 그려 보라"**입니다. 통계 숫자만으로는 알 수 없는 것이 그림에서는 한눈에 보입니다. 4-3 절의 산점도가 없었다면 "숨은 이상 17건이 톤·거칠기 축에서는 뚜렷한 무리를 이룬다"는 사실을 알아채기 어려웠을 것입니다.

IsolationForest 원리

이상은 고립시키기 쉽다는 한 문장

5-1. 발상의 전환 — 정상을 그리지 않습니다

Z-score는 **정상의 모습을 먼저 그리고**, 거기서 얼마나 멀어졌는지를 잹습니다. IsolationForest는 정반대로 갑니다 — **정상을 그리지 않고 곧장 이상을 떼어냅니다**.

	Z-score	IsolationForest
1단계	정상의 평균·표준편차를 구함	데이터를 무작위로 계속 자름
2단계	각 값이 몇 칸 떨어졌는지 계산	각 점이 혼자가 될 때까지 몇 번 잘랐는지 셈
판정	칸 수가 크면 이상	자른 횟수가 적으면 이상
보는 범위	특징 하나씩	여러 특징을 한꺼번에

개념

"**이상은 고립시키기 쉽다**" — 이 한 문장이 알고리즘의 심장입니다. 무리에서 동떨어진 점은 아무렇게나 잘라도 금방 혼자가 되고, 무리 한가운데 있는 점은 촘촘히 여러 번 잘라야 겨우 혼자가 됩니다. **그 횟수 차이로 이상을 가릅니다**.

5-2. 스무고개 비유 — 특이할수록 빨리 좁혀집니다

교안의 비유가 아주 좋습니다. 스무고개를 한다고 생각해 보십시오.

정답	질문 흐름	필요한 질문 수
사과	과일인가? → 빨간가? → 둥근가? → 배인가? 감인가? ...	많이 필요 — 비슷한 게 많음
에펠탑	사람이 만든 건가? → 아주 높은가?	두세 개면 끝 — 워낙 특이함

개념

사과는 비슷한 과일이 많아서 질문을 많이 던져야 하나로 좁혀집니다. 반면 **에펠탑은 워낙 특이해서** 두세 질문이면 꼭 집힙니다. 데이터도 똑같습니다 — **무리 속 정상은 고립시키기 어렵고, 무리 밖 이상은 쉽습니다**.

5-3. 트리 — 예·아니오로 계속 둘로 나누기

여기서 **트리(tree)**라는 말이 나옵니다. 이름이 거창하지만 하는 일은 단순합니다 — **예·아니오** 질문을 던져 데이터를 **둘로 계속 나누는** 구조입니다.



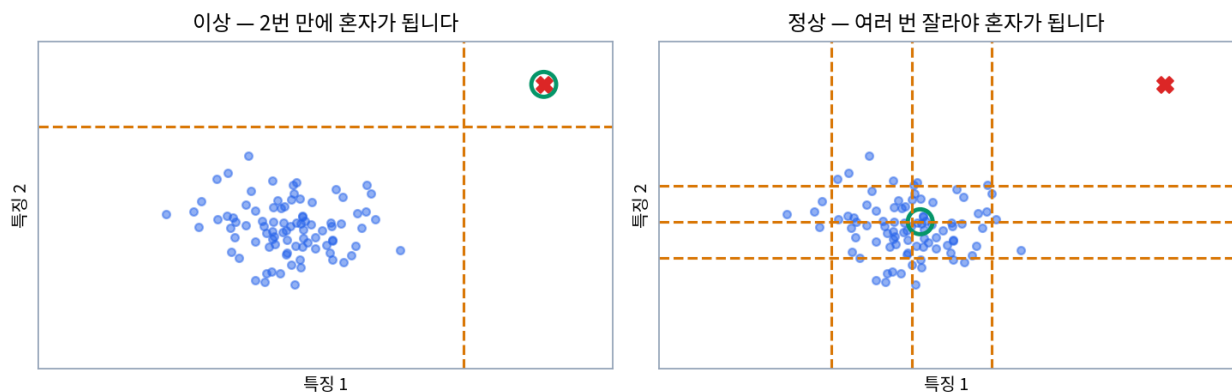
용어	뜻
트리 (tree)	질문을 던져 데이터를 둘로 계속 나누는 구조 — 나무 한 그루
분할 (split)	질문 하나로 데이터를 둘로 가르는 행위
고립 (isolation)	분할을 반복해 어떤 점이 혼자만 남는 칸에 도달하는 것
고립 깊이	혼자가 되기까지 거친 분할의 횟수
포레스트 (forest)	나무 여러 그루 — 한 그루의 운을 평균으로 다스림

개념

"혼자가 되기까지 걸린 질문 수"가 이상 판정의 단서입니다. 정상은 비슷한 친구가 많아 한참 쪼개야 겨우 혼자가 되고, 이상은 무리에서 동떨어져 몇 번 만에 똑 혼자가 됩니다.

말보다 그림이 빠릅니다. 아래 두 그림에서 **주황 점선이 분할선**이고, **초록 동그라미가 목표로 삼은 점**입니다.

IsolationForest의 핵심 — 이상은 적은 분할로, 정상은 많은 분할로 고립됩니다



▲ 왼쪽 — 구석의 이상(빨간 X)은 선 두 개만 그으면 자기 영역에 혼자 남습니다. 오른쪽 — 무리 한가운데 정상은 여섯 번을 잘라도 아직 친구들과 같은 칸에 있습니다

개념

왼쪽 그림을 보십시오. 세로선 하나와 가로선 하나, **총 두 번만** 그으면 오른쪽 위 영역에 빨간 X 혼자 남습니다. 반면 **오른쪽 그림에서는** 여섯 번을 잘랐는데도 초록 동그라미 주변에 파란 점들이 여전히 함께 있습니다. **이 횟수 차이가 곧 이상 점수가 됩니다.**

5-4. 무작위로 자르는데 왜 되나요

여기서 자연스러운 의문이 생깁니다. **어디를 자를지 똑똑하게 계산하지 않고 무작위로 자르는데도 잘 작동할까요?**

단계	무엇을 무작위로 정하나	예시
① 특징 선택	여러 특징 중 하나를 무작위로 고름	rms를 고름
② 분할 지점	그 특징의 값 범위 안에서 자를 위치를 무작위로 고름	0.072에서 자름
③ 반복	나뉜 각 무리에서 ①②를 혼자 남을 때까지 반복	—

개념

답은 **"이상이 워낙 동떨어져 있어서"**입니다. 무리에서 멀리 떨어진 점은 **아무 데나 잘라도** 금방 자기 혼자만의 영역에 갇힙니다. 반대로 무리 한가운데 있는 점은 아무리 잘라도 주변에 친구가 남아 있습니다. **똑똑하게 계산할 필요 없이 무작위로도 충분하고, 계산하지 않으니 훨씬 빠릅니다.**

이득	내용
속도	특징과 분할점을 계산하지 않으니 매우 빠름 — 특징이 많아져도 부담이 적음
거리 계산 불필요	점 사이 거리를 직접 재지 않고 분할 횟수만 세도 거리감과 비슷한 정보를 얻음

주의

다만 한 그루는 운에 좌우됩니다. 운 나쁘게 잘리면 정상이 빨리 고립되기도 하고 이상이 늦게 고립되기도 합니다. 그래서 **여러 그루를 만들어 고립 깊이의 평균**을 냅니다. 이것이 이름에 **Forest(숲)**가 붙은 이유입니다 — 여러 심사위원의 평균 점수가 공정하듯, 우연이 묻히고 안정적인 값이 남습니다.

5-5. 고립 깊이가 이상 점수가 됩니다

고립 깊이를 그대로 쓰면 "몇 번"이라는 숫자로 다루기 불편합니다. 그래서 **0~1 사이의 점수로 환산**합니다.

고립 깊이	뜻	개념상 이상 점수
얕음 (2~3회)	금방 혼자가 됨 — 이상	1에 가까움
깊음 (여러 번)	한참 잘라야 혼자가 됨 — 정상	0에 가까움

주의

여기서 아주 헷갈리는 지점이 하나 있습니다. 개념상으로는 "1에 가까울수록 이상"인데, 우리가 쓸 `scikit-learn`의 점수 함수는 **방향이 반대**입니다 — **작을수록 이상**입니다. CH 6-4절에서 다시 강조하겠지만, 미리 "**sklearn 점수는 작을수록 이상**"이라고 외워 두시는 편이 안전합니다.

점수가 있으면 좋은 점은 **정도까지 알 수 있다**는 것입니다. "이상 40건입니다"가 아니라 "**가장 이상한 순서로 1번부터 40번까지**"를 줄 수 있습니다.

5-6. scikit-learn — 새로 등장하는 다섯 번째 도구

이제 도구를 만납니다. `scikit-learn`(**줄여서 sklearn**)은 파이썬에서 가장 널리 쓰이는 머신러닝 라이브러리입니다. 지금까지 쓰던 네 도구와 성격이 다릅니다.

도구	하는 일	비유
pandas	표를 다룸	데이터 그릇
numpy	숫자 계산	계산 엔진
matplotlib·seaborn	그림 그리기	붓
scikit-learn	모델을 만들고 학습시키고 예측함	새로 등장

sklearn의 좋은 점은 **모든 모델이 똑같은 세 단계로 쓰인다**는 것입니다. 오늘 `IsolationForest`를 익혀 두면, 나중에 다른 모델을 만나도 흐름이 같습니다.



단계	메서드	하는 일	비유
① 만들기	<code>IsolationForest(...)</code>	설정값만 갖춘 빈 모델을 하나 만듦	빈 공책을 준비
② 학습	<code>.fit(X)</code>	데이터를 보여 주며 숲을 실제로 지음 — 보통 한 번만	공책에 내용을 채움
③ 예측	<code>.predict(X)</code>	학습된 모델로 각 행을 판정	공책을 보고 답함

주의

`fit()`에는 `label`을 넣지 않습니다. 넣는 것은 특징 세 개뿐입니다. 이상탐지는 비지도학습이라 정답 없이 학습하기 때문입니다(0-3점). 정답을 넣는 것은 지도학습이고, 그때는 `.fit(X, y)`처럼 인자가 두 개가 됩니다.

개념

`fit`과 `predict`를 왜 나눠 뒀을까요. 실제 운영이 "한 번 학습, 여러 번 예측"이기 때문입니다. 숲을 짓는 것은 오래 걸리지만 예측은 순식간이라, 한 번 지어 두고 새 측정값이 들어올 때마다 `predict`만 반복합니다. CH 7-4절에서 이것을 직접 해 봅니다.

5-7. 코드 세 줄

길게 배운 고립과 숲의 원리가 딱 세 줄로 압축됩니다.

IsolationForest 첫 적용

```
from sklearn.ensemble import IsolationForest

# 특징 세 개만 골라 X로 (label은 넣지 않음)
X = df[["rms", "spectral_centroid", "zero_crossing_rate"]]

iso = IsolationForest(contamination=0.18, random_state=42) # ① 만들기
iso.fit(X) # ② 학습
pred = iso.predict(X) # ③ 예측

print("predict 결과에 들어 있는 값:", sorted(set(pred)))
```

▶ predict 결과에 들어 있는 값: [-1, 1]

주의

`from sklearn.ensemble import IsolationForest` 라는 줄이 낫설 수 있습니다. `sklearn` 안의 `ensemble`이라는 서랍에서 `IsolationForest`만 꺼내 온다는 뜻입니다. `import pandas as pd`처럼 통째로 가져오는 대신 필요한 것 하나만 가져오는 방식이고, `sklearn`은 워낙 커서 보통 이렇게 씁니다.

5-8. contamination — 가장 중요한 설정값

코드에 `contamination=0.18`이라는 값이 있었습니다. 이것이 `IsolationForest`의 임계값에 해당합니다.

항목	내용
뜻	오염도 — 전체 데이터 중 이상이 차지하는 대략의 비율

항목	내용
0.18의 의미	점수가 가장 이상한 쪽 상위 약 18%를 이상으로 표시하라
왜 0.18인가	실습 데이터가 220개 중 이상 40개 = 18.2%이므로 (0-4절)
트레이드오프	키우면 많이 잡고 오탐 높 · 줄이면 적게 잡고 놓침 높 — 임계값과 똑같은 시소

주의

현장에서는 정확한 이상 비율을 모릅니다. 그래서 실습처럼 정답을 보고 0.18을 넣는 것은 사실 반칙에 가깝습니다. 실제로는 작게 시작해서(0.05 정도) 결과를 보며 조정합니다. CH 7-1절에서 값을 바꿔 가며 실험합니다.

5-9. n_estimators와 random_state

설정값이 두 개 더 있습니다. 둘 다 오늘은 기본값으로 두지만, 무엇을 하는지는 알아 두셔야 합니다.

설정값	역할	기본값	올리면
n_estimators	안정성	100	나무가 많아져 결과가 안정 — 대신 느려짐
random_state	재현성	없음	고정하면 몇 번을 돌려도 항상 같은 결과

개념

random_state가 "마법 주사위"입니다. 진짜 주사위는 굴릴 때마다 다른 눈이 나오지만, random_state=42를 넣으면 항상 같은 순서로 같은 눈이 나옵니다. 5-4절에서 배운 대로 이 알고리즘은 무작위로 자르기 때문에, 고정하지 않으면 돌릴 때마다 결과가 조금씩 달라집니다. 실습에서 42로 고정하는 이유가 이것입니다.

주의

42라는 숫자에 의미는 없습니다. 관습적으로 많이 쓰는 값일 뿐이고 0이든 7이든 상관없습니다. 중요한 것은 팀 안에서 같은 값을 쓰기로 약속하는 것입니다. 값이 다르면 같은 코드인데 결과가 달라져서 "왜 내 것만 다르지" 하는 일이 생깁니다.

5-10. 학습 데이터에 이상이 섞여도 되나요

CH 4-5절에서 Z-score의 가장 큰 약점이 기준선 오염이었습니다. IsolationForest는 이 문제에서 훨씬 자유롭습니다.

	Z-score	IsolationForest
이상이 조금 섞이면	평균·표준편차가 오염 — 탐지력 붕괴	견딜 수 있음 — 이상은 어차피 소수라 빨리 고립됨
왜 그런가	평균·표준편차에 직접 의존	평균을 아예 쓰지 않음

	Z-score	IsolationForest
현장 이점	정상만 깔끔히 골라내야 함	그 부담이 줄어들

주의

다만 "소수가 섞인 정도"까지만 견딥니다. 이상이 절반 가까이 되면 전제가 깨져서 제대로 작동하지 않습니다 — 이상이 다수면 그게 더 이상 이상이 아니기 때문입니다. 보통 설비처럼 **정상이 대부분일 때** 쓰기 좋은 도구입니다.

5-11. Z-score와 나란히 놓고 보기

CH 5에서 배운 것을 CH 1~4의 Z-score와 **같은 항목으로 나란히** 놓으면 두 도구의 성격 차이가 분명해집니다.

질문	Z-score의 답	IsolationForest의 답
정상을 어떻게 표현하나	평균과 표준편차 두 숫자로	표현하지 않음 — 이상을 직접 떼어냄
거리를 어떻게 재나	평균에서 몇 표준편차인지 계산	몇 번 잘라야 혼자가 되는지 셈
설정값은 무엇인가	임계값 하나 (2 · 3 · 자유)	<code>contamination · n_estimators · random_state</code>
탐지 개수는 어떻게 정해지나	임계값을 넘는 것이 몇 개든 그만큼	contamination이 개수를 직접 정함
이상이 섞이면	기준선이 오염되어 무너짐	어느 정도 견딤

개념

네 번째 행이 실무에서 큰 차이를 만듭니다. Z-score는 "임계값을 넘는 것이 0개일 수도, 100개일 수도" 있습니다 — 데이터가 정합합니다. 반면 IsolationForest는 **contamination이 개수를 강제로 정합니다** — 220개에 0.18을 넣으면 이상이 하나도 없어도 40개를 골라냅니다. **정상만 있는 날에도 40개가 나온다는** 뜻이라, 이것이 오탐의 근본 원인이 됩니다.

주의

그래서 IsolationForest 결과를 볼 때는 개수보다 점수를 봐야 합니다. "40건이 이상입니다"가 아니라 "점수가 -0.6 아래인 것이 12건입니다"처럼 점수 기준으로 읽으면, 정상만 있는 날에는 그 12건이 0건이 됩니다. CH 6-4 절에서 점수를 다루는 이유가 이것입니다.

모델 적용과 결과 해석

-1과 1의 함정, 그리고 이상 점수

6-1. predict가 돌려주는 -1과 1

5-7절에서 `pred = iso.predict(X)`를 돌렸더니 결과에 **-1과 1** 두 종류만 들어 있었습니다. 이것이 **초보자가 가장 많이 헷갈리는 지점**입니다.

scikit-learn의 약속	우리가 익숙한 방식
이상 = -1 · 정상 = 1	이상 = 1 · 정상 = 0

주의

정반대입니다. MIMII 데이터의 `label`은 이상이 1인데 sklearn의 `predict`는 이상이 -1입니다. **모르고 그대로 비교하면 정상과 이상을 통째로 뒤집어 해석하게 됩니다.** 정확도가 이상하게 낮게 나오면 가장 먼저 의심할 지점입니다.

개념

외우는 법이 있습니다. 통계에서 무리 밖의 점을 **outlier**, 무리 안의 점을 **inlier**라고 부릅니다. **바깥(out)이 -1, 안(in)이 1**이라고 기억하시면 헷갈리지 않습니다.

6-2. 우리 형식으로 바꾸기

패턴 -1/1 → 1/0 변환 sklearn 결과를 우리 라벨 기준에 맞추기

정의 `pred == -1`이라는 조건식을 만들면 이상인 곳만 True가 되고, 여기에 `.astype(int)`를 붙이면 1/0이 됩니다.

형태 `df["if_anom"] = (pred == -1).astype(int)`

```

from sklearn.ensemble import IsolationForest

F = ["rms", "spectral_centroid", "zero_crossing_rate"]
X = df[F]

iso = IsolationForest(contamination=0.18, random_state=42)
iso.fit(X)
pred = iso.predict(X)

# sklearn 형식(-1/1)을 우리 형식(1/0)으로
df["if_anom"] = (pred == -1).astype(int)

print("이상으로 잡힌 개수:", int(df["if_anom"].sum()), "건")

```

▶ 이상으로 잡힌 개수: 40 건

왜 쓰나 정확히 40건입니다. $\text{contamination}=0.18$ 이고 전체가 220개니 $220 \times 0.18 = 39.6 \rightarrow 40$ 건이 나옵니다.

즉 **contamination이 탐지 개수를 직접 정합니다** — Z-score의 임계값이 "몇 점 넘으면"이었다면, contamination은 "상위 몇 %"입니다.

응용 변환이 제대로 됐는지 확인하는 습관을 들이면 좋습니다. 개수를 세어 예상과 맞는지 보면 됩니다.

```

import numpy as np
print("pred 값 분포:", dict(zip(*np.unique(pred, return_counts=True))))
print("변환 후 합계:", int(df["if_anom"].sum()), " (= -1의 개수와 같아야 함)")

```

▶ pred 값 분포: {np.int64(-1): np.int64(40), np.int64(1): np.int64(180)}

▶ 변환 후 합계: 40 (= -1의 개수와 같아야 함)

개념

-1이 40개, 1이 180개입니다. 우연히도 실제 label 분포(이상 40, 정상 180)와 개수가 같은데, **개수가 같다고 같은 40개는 아닙니다**. 어느 40개를 골랐는지가 진짜 성능이고, 그것을 다음 절의 교차표로 확인합니다.

자주 하는 실수

변환 방향을 거꾸로 해서 ($\text{pred} == 1$)로 쓰면 **정상 180개가 이상으로 표시**됩니다. 그러면 그 뒤 모든 결과가 뒤집힙니다. **변환 직후 개수를 세어 확인하는 것이 이 실수를 막는 가장 확실한 방법**입니다.

6-3. 교차표로 채점하기

이제 label을 꺼내 채점합니다(2-3절의 "채점할 때만 답안지를 펼친다"). **교차표**를 쓰면 네 가지 경우를 한눈에 볼 수 있습니다.

함수 `pd.crosstab()` 두 컬럼을 교차해 개수를 세는 표

정의 첫 인자를 행으로, 둘째 인자를 열로 놓고 **각 조합에 몇 개가 있는지** 세어 표로 만듭니다.

형태 `pd.crosstab(행이 될 컬럼, 열이 될 컬럼)`

```
ct = pd.crosstab(df["label"], df["if_anom"])
print(ct.to_string())
```

```
▶ if_anom    0    1
▶ label
▶ 0          166   14
▶ 1          14   26
```

왜 쓰나 네 칸이 각각 다른 의미를 가집니다. 행은 정답(label), 열은 모델 판정(if_anom)입니다. 왼쪽 위부터 시계 방향으로 정상 맞춤 · 거짓 경보 · 이상 맞춤 · 놓침입니다.

응용 숫자만으로는 헷갈리니 이름을 붙여 읽어 보겠습니다.

```
tn, fp = int(ct.loc[0,0]), int(ct.loc[0,1])
fn, tp = int(ct.loc[1,0]), int(ct.loc[1,1])
```

```
print(f"정상 맞춤 {tn:3}건 (정상인데 정상이라 함)")
print(f"거짓 경보 {fp:3}건 (정상인데 이상이라 함)")
print(f"놓침 {fn:3}건 (이상인데 정상이라 함)")
print(f"이상 맞춤 {tp:3}건 (이상인데 이상이라 함)")
print()
print(f"재현율 = {tp}/{tp+fn} = {tp/(tp+fn)*100:.1f}%")
print(f"정밀도 = {tp}/{tp+fp} = {tp/(tp+fp)*100:.1f}%")
```

```
▶ 정상 맞춤 166건 (정상인데 정상이라 함)
▶ 거짓 경보 14건 (정상인데 이상이라 함)
▶ 놓침 14건 (이상인데 정상이라 함)
▶ 이상 맞춤 26건 (이상인데 이상이라 함)
▶
▶ 재현율 = 26/40 = 65.0%
▶ 정밀도 = 26/40 = 65.0%
```

개념

실제 이상 40건 중 26건을 잡았습니다. 재현율 65%이고, 잡은 40건 중 26건이 진짜였으니 정밀도도 65%입니다. 33일차에 배운 두 지표가 여기서 그대로 쓰입니다.

설비 적용

Z-score의 rms 단독(23건)과 비교하면 3건이 늘었습니다. 그런데 오탐이 0건에서 14건으로 늘었습니다. CH 7-5 절에서 세 방법을 나란히 놓고 비교하는데, 이 트레이드오프가 그때 분명해집니다.

자주 하는 실수

교안은 이 자리에서 "약 27개"라고 적고 있는데, 저희가 실제로 돌린 결과는 26개입니다. contamination=0.18 이 220개의 18%인 39.6개를 40개로 반올림하는 과정에서 **경계선에 걸린 한 건이 사이킷런 버전에 따라 갈릴 수 있습니다.** 교안 숫자와 1건 차이가 나도 놀라지 않으셔도 됩니다.

6-4. 이상 점수 — 얼마나 이상한가

predict는 -1이냐 1이냐만 알려 줍니다. **얼마나 이상한지**는 알려 주지 않습니다. 그것을 알려면 **점수**를 봐야 합니다.

메서드 `.score_samples()` 각 행의 이상 점수를 숫자로

정의 학습된 모델이 각 행에 매긴 이상 점수를 돌려줍니다. **값이 작을수록 이상**입니다.

형태 `iso.score_samples(X)`

```
df["score"] = iso.score_samples(X)

print(f"점수 범위: {df['score'].min():.4f} ~ {df['score'].max():.4f}")
print()
print("가장 이상한 5개 (점수가 작은 순)")
print(df.nsmallest(5, "score")[F + ["score", "label"]].round(4).to_string())
```

▶ 점수 범위: -0.6785 ~ -0.3715

▶

▶ 가장 이상한 5개 (점수가 작은 순)

	rms	spectral_centroid	zero_crossing_rate	score	label
▶ 70	0.1599	1947.0	0.0891	-0.6785	1
▶ 54	0.0144	1864.1	0.1384	-0.6388	0
▶ 108	0.1467	2674.4	0.1037	-0.6253	1
▶ 157	0.1273	1577.0	0.0811	-0.6136	1
▶ 149	0.1303	2126.7	0.1419	-0.6109	1

왜 쓰나 1순위가 70번인데 Z-score에서도 1순위였습니다(3-3절, Z=9.006). 서로 다른 원리의 두 도구가 같은 결론을 냈다는 뜻이라, 그만큼 믿을 만합니다.

응용 2순위인 54번이 흥미롭습니다. label이 0(정상)인데 모델은 아주 이상하다고 판정했습니다. 특징값을 보면 이유를 알 수 있습니다.

```
normal = df[df.label == 0]
row = df.loc[54]
for c in F:
    z = (row[c] - normal[c].mean()) / normal[c].std()
    print(f"{c:20} {row[c]:9.4f}    Z {z:6.2f}")
```

```
▶ rms                0.0144    Z  -2.91
▶ spectral_centroid  1864.1000    Z  -0.48
▶ zero_crossing_rate  0.1384    Z   2.50
```

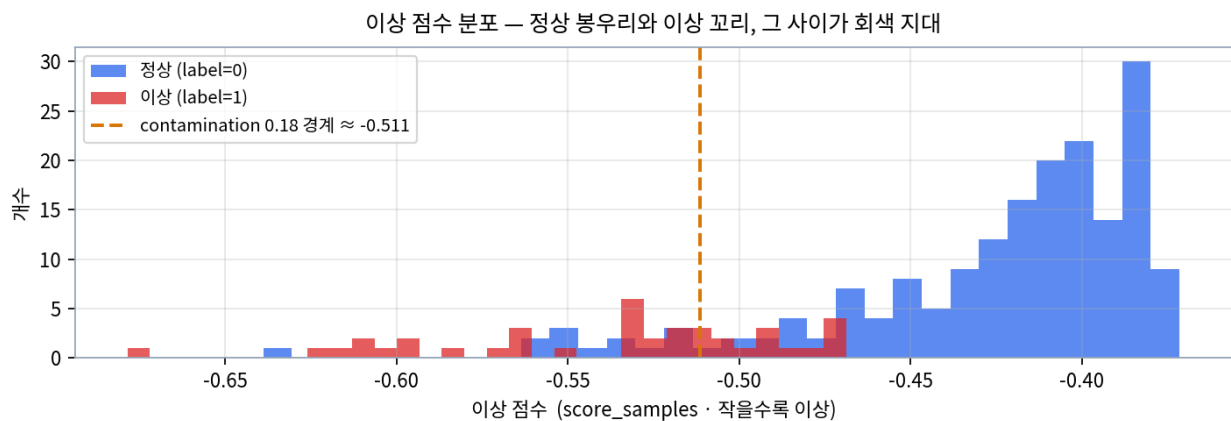
개념

54번은 rms가 아주 작고(Z -2.91) 거칠기는 큼니다(Z +2.50). 소리가 약한데 거칠다는 조합인데, 이것은 정상 기계에서 잘 안 나오는 조합입니다. **label은 정상이라고 하지만 모델 입장에서는 충분히 의심스러운 점입니다** — 4-2절에서 말한 "조합이 특이한 경우"의 실제 사례입니다.

자주 하는 실수

부호 방향을 다시 강조합니다. 5-5절에서 "개념상 점수는 1에 가까울수록 이상"이라고 했는데, score_samples는 **작을수록(음수 쪽으로 갈수록) 이상**입니다. 오늘 점수 범위가 -0.68 ~ -0.37인데, **-0.68이 가장 이상**하고 -0.37이 가장 정상입니다.

점수의 분포를 그림으로 보면 정상과 이상이 어떻게 갈리는지가 한눈에 들어옵니다.



▲ 이상 점수 분포 — 정상(파랑)은 오른쪽에 봉우리를 이루고, 이상(빨강)은 왼쪽 꼬리로 흩어집니다. 주황 점선이 contamination 0.18이 정한 경계선입니다

개념

그림에서 세 가지를 보시면 됩니다. ① 파란 봉우리가 -0.45 근처에 몰려 있고, ② 빨간 점들이 왼쪽으로 길게 꼬리를 그리며, ③ 주황 점선(경계)이 그 사이를 자릅니다. 점선 왼쪽이 이상으로 판정된 40건입니다. 그런데 점선 근처에 파랑과 빨강이 섞여 있는 구간이 보이는데, 이곳이 바로 오탐 14건과 놓침 14건이 생기는 자리입니다 — 이 구간을 회색 지대라고 부릅니다.

6-5. 점수로 우선순위 매기기

점수가 있으면 가장 이상한 순서로 정렬할 수 있습니다. 이것이 실무에서 점수가 필요한 진짜 이유입니다.

점검 우선순위 Top 10

```
top10 = df.nsmallest(10, "score")
print(top10[F + ["score", "label"]].round(4).to_string())
print()
print("상위 10개 중 실제 이상:", int(top10["label"].sum()), "/ 10")
```

```
►          rms  spectral_centroid  zero_crossing_rate  score  label
► 70    0.1599             1947.0             0.0891 -0.6785      1
► 54    0.0144             1864.1             0.1384 -0.6388      0
► 108   0.1467             2674.4             0.1037 -0.6253      1
► 157   0.1273             1577.0             0.0811 -0.6136      1
► 149   0.1303             2126.7             0.1419 -0.6109      1
► 81    0.0757             3055.9             0.1451 -0.6088      1
► 159   0.0670             2703.1             0.1722 -0.6008      1
► 23    0.1250             2846.4             0.0687 -0.5988      1
► 90    0.1259             2910.5             0.1078 -0.5976      1
► 218   0.1247             2783.3             0.0794 -0.5838      1
►
► 상위 10개 중 실제 이상: 9 / 10
```

샘플	왜 이상한가	유형
70번	rms 0.1599 — 세기가 압도적으로 큼	명백한 이상
54번	rms 0.0144로 아주 작는데 거칠기는 높음	조합 이상 (label은 정상)
81번	rms는 0.0757로 평범한데 톤 3055.9 · 거칠기 0.1451	다변량 이상 — Z-score가 놓친 것
159번	rms 0.0670 평범, 톤 2703 · 거칠기 0.1722	다변량 이상

개념

상위 10개 중 9개가 실제 이상입니다. 그리고 그중 81번 · 159번은 Z-score의 rms 단독으로는 못 잡았던 다변량 이상입니다(3-7절 목록에 둘 다 있습니다). IsolationForest가 여러 특징을 동시에 본다는 것이 여기서 실제로 확인됩니다.

설비 적용

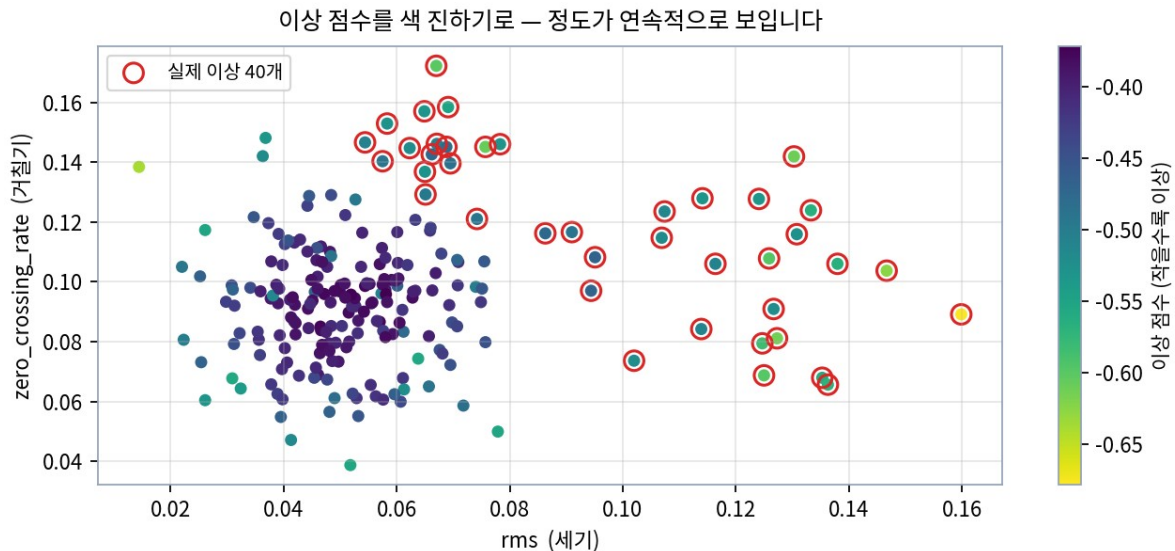
"40개가 이상입니다"보다 "1순위는 70번입니다"가 현장에서 받는 무게가 다릅니다. 정비 인력은 한정되어 있으니 40개를 한 번에 볼 수 없습니다. 점수 순으로 상위 3개 또는 10개만 뽑아 주면 실제로 움직일 수 있습니다.

주의

다만 점수는 "정도"이지 "심각도"가 아닙니다. 점수가 낮다는 것은 정상 무리에서 멀다는 뜻이지 위험한 고장이라는 보장은 없습니다. 그래서 순위는 참고로 쓰되, **특징값을 함께 보여 줘야** 정비사가 무엇을 점검할지 가늠할 수 있습니다 — 위 표의 "왜 이상한가" 열이 그 역할입니다.

6-6. 점수를 색으로 — 정도를 연속적으로 보기

점수를 산점도에 색의 진하기로 입히면, 정상과 이상 두 색으로 칠하는 것보다 훨씬 많은 정보가 보입니다.



▲ 점 색깔이 이상 점수 — 진할수록 이상합니다. 빨간 빈 원이 실제 이상 40건인데, 오른쪽 끝(세기 큼)과 위쪽(거칠기 큼)에 진한 점이 몰려 있습니다

개념

두 색으로 칠하면 "이상이나 아니냐"만 보이지만, 색 농담으로 칠하면 "얼마나 이상하냐"가 연속적으로 보입니다. 그래서 **회색 지대**가 드러납니다 — 오른쪽 아래에서 왼쪽 위로 가면서 색이 서서히 진해지는 구간이 있는데, 그곳이 정상과 이상이 겹치는 자리입니다.

주의

그림을 보면 빨간 원 중에도 색이 옅은 것들이 있습니다. 실제로는 이상인데 모델이 별로 이상하지 않다고 본 점들이고, 이것이 **농친 14건**입니다. 반대로 빨간 원이 없는데 색이 진한 점들은 **오탐 14건**입니다. 6-4절 히스토그램에서 본 회색 지대가 산점도에서는 이렇게 나타납니다.

파라미터와 활용

다이얼을 돌려 보고, 두 도구를 나란히 놓기

7-1. contamination을 바꾸면 어떻게 달라지나

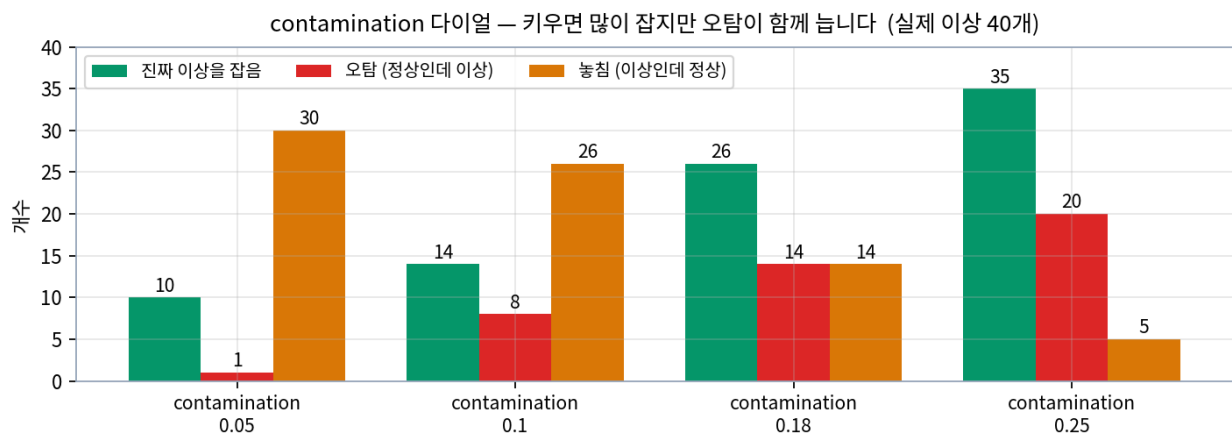
5-8절에서 contamination이 "상위 몇 %를 이상으로 볼지"를 정한다고 했습니다. 값을 바꿔 가며 직접 확인해 보겠습니다.

contamination 실험

```
for c in [0.05, 0.10, 0.18, 0.25]:
    m = IsolationForest(contamination=c, random_state=42).fit(X)
    p = (m.predict(X) == -1).astype(int)

    tp = int(((p == 1) & (df.label == 1)).sum()) # 진짜 이상을 잡음
    fp = int(((p == 1) & (df.label == 0)).sum()) # 오탐
    fn = int(((p == 0) & (df.label == 1)).sum()) # 놓침
    print(f"contamination {c:<5} 잡힌 {int(p.sum()):3}개 · "
          f"진짜 {tp:3} 오탐 {fp:3} 놓침 {fn:3}")
```

- ▶ contamination 0.05 잡힌 11개 · 진짜 10 오탐 1 놓침 30
- ▶ contamination 0.1 잡힌 22개 · 진짜 14 오탐 8 놓침 26
- ▶ contamination 0.18 잡힌 40개 · 진짜 26 오탐 14 놓침 14
- ▶ contamination 0.25 잡힌 55개 · 진짜 35 오탐 20 놓침 5



▲ contamination을 키울수록 진짜 이상(초록)과 오탐(빨강)이 함께 늘고 놓침(주황)은 줄어듭니다 — 1-7절의 시소가 그대로 재현됩니다

contamination	잡힌 개수	진짜 이상	오탐	성격
0.05	11개	10개	1개	정확하지만 30건을 놓침
0.10	22개	14개	8개	—
0.18	40개	26개	14개	균형 — 실제 비율에 맞춤
0.25	55개	35개	20개	많이 잡지만 거짓 경보 20건

개념

0.05는 정확도가 압도적입니다 — 11개 잡아서 10개가 진짜, 오탐은 1개뿐입니다. 하지만 **30건을 놓칩니다**. 반대로 0.25는 35건을 건지지만 오탐이 20건입니다. 어느 쪽이 나은지는 **미탐 비용과 오탐 비용의 비율**에 달려 있습니다 — 33일차 4-7절의 비대칭비 이야기입니다.

주의

contamination을 바꿔도 이상 점수의 순위 자체는 변하지 않습니다. 점수는 그대로이고 어디까지를 이상으로 자를지 경계만 옮겨집니다. 그래서 0.05에서 잡힌 11개는 0.25에서도 반드시 잡힙니다 — 점수 상위 11개니까요. 5-3절에서 배운 "임계값 조정과 모델 개선은 다르다"와 같은 이야기입니다.

설비 적용

현장에서는 정답 비율을 모릅니다. 우리가 0.18을 넣을 수 있었던 것은 label을 봤기 때문인데, 실제로는 그럴 수 없습니다. 그래서 **작게 시작해 결과를 보며 조정합니다** — 0.05로 시작해서 잡힌 11건을 현장이 확인해 보고, 진짜가 많으면 조금 올리고 헛걸음이 많으면 내리는 식입니다.

7-2. 스케일링은 필요할까 — StandardScaler

데이터 분석에서 **전처리**로 흔히 쓰는 것이 StandardScaler입니다. 오늘 세 특징은 크기가 제각각($0.05 \cdot 1979 \cdot 0.091$)이라 맞춰 줘야 할 것 같은데, 정말 그럴까요.

개념 StandardScaler 특징마다 다른 값 범위를 맞추는 전처리

정의 각 특징을 **평균 0, 표준편차 1**로 바꿉니다. 사실 **Z-score**와 똑같은 계산입니다 — $(\text{값} - \text{평균}) / \text{표준편차}$.

```

from sklearn.preprocessing import StandardScaler

# ① 원본으로 학습·예측 (비교 기준)
m1 = IsolationForest(contamination=0.18, random_state=42).fit(X)
p1 = (m1.predict(X) == -1).astype(int)

# ② 스케일링 후 학습·예측
X_scaled = StandardScaler().fit_transform(X)
m2 = IsolationForest(contamination=0.18, random_state=42).fit(X_scaled)
p2 = (m2.predict(X_scaled) == -1).astype(int)

print("원본 진짜 이상 잡음:", int(((p1==1) & (df.label==1)).sum()), "건")
print("스케일 진짜 이상 잡음:", int(((p2==1) & (df.label==1)).sum()), "건")
print("두 결과가 일치하는 행:", int((p1 == p2).sum()), "/ 220")

```

▶ 원본 진짜 이상 잡음: 26 건

▶ 스케일 진짜 이상 잡음: 26 건

▶ 두 결과가 일치하는 행: 220 / 220

왜 쓰나 220행 전부가 똑같습니다. 스케일링을 해도 결과가 한 톨도 안 바뀌었습니다. 값 범위가 그렇게 달랐는데도 말입니다.

응용 왜 그런지는 5-3절의 트리 원리를 떠올리면 이해됩니다. 트리는 "이 특징이 이 값보다 큰가 작은가"만 묻습니다.

```

# 스케일링은 값을 옮기지만 순서는 바꾸지 않는다
a = X["spectral_centroid"].head(5).values
b = X_scaled[:5, 1]
for i in range(5):
    print(f"원본 {a[i]:8.1f} → 스케일 {b[i]:6.3f}")

```

▶ 원본 1982.7 → 스케일 -0.267

▶ 원본 2160.8 → 스케일 0.277

▶ 원본 2065.9 → 스케일 -0.013

▶ 원본 1884.4 → 스케일 -0.568

▶ 원본 2430.9 → 스케일 1.103

개념

값은 완전히 달라졌지만 크기 순서는 그대로입니다. 원본이 $1884.4 < 1982.7 < 2065.9 < 2160.8 < 2430.9$ 였는데, 스케일 후에도 $-0.568 < -0.267 < -0.013 < 0.277 < 1.103$ 으로 순서가 똑같습니다. 트리는 순서만 보니 어느 쪽을 써도 똑같은 자리에서 자릅니다.

설비 적용

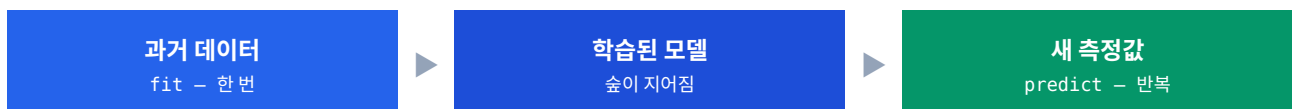
그렇다고 **StandardScaler**가 쓸모없는 것은 아닙니다. 점 사이 거리를 직접 재는 방법(K-평균, KNN 등)에서는 거의 필수입니다. 값 범위가 1979인 특징이 0.05인 특징을 압도해 버리기 때문입니다. **트리 기반에만 영향이 작은 것**입니다.

자주 하는 실수

좋다고 알려진 전처리도 모델에 따라 효과가 다릅니다. 무작정 넣지 말고 **적용 전후를 비교해** 우리 데이터·모델에서 **효과를 확인하고 판단**하는 것이 정석입니다. 오늘 실습이 정확히 그 확인 절차였습니다.

7-3. 한 번 학습, 여러 번 예측

5-6절에서 fit과 predict를 왜 나눠 뒀는지 물었습니다. 답이 여기 있습니다.



단계	언제	속도	왜
fit	한 번 (또는 가끔 갱신)	느림	나무 100그룹을 실제로 지어야 함
predict	새 값이 들어올 때마다	매우 빠름	이미 지어진 나무를 따라 내려가기만 하면 됨

개념

이 구조가 **실시간 모니터링의 핵심**입니다. 매번 다시 학습하면 느려서 실시간이 불가능하고, 게다가 **기준이 계속 바뀌어 일관성이 없어집니다**. 한 번 지어 둔 기준으로 계속 보니까 "어제와 오늘을 같은 잣대로 비교"할 수 있습니다.

주의

새 데이터로 다시 학습하면 안 됩니다. 새 데이터에 이상이 많이 섞여 있으면 그것을 정상으로 배워 버립니다. fit은 믿을 수 있는 과거 데이터로 한 번, 그 뒤로는 predict만 반복하는 것이 원칙입니다.

7-4. 새 데이터에 적용 — label 없는 상황

이제 실제 현장과 같은 상황을 만들어 봅니다. 26_mimii_new.csv에는 label이 없습니다(0-4절). **정답 없이 모델이 이상을 가려낼 수 있는지 확인**합니다.

새 데이터 40건에 적용

```
new = pd.read_csv("26_mimii_new.csv")
print("새 데이터:", new.shape, "\t 컬럼:", list(new.columns))

# 학습 때와 같은 특징을 같은 순서로
X_new = new[F]

pred_new = iso.predict(X_new)
new["score"] = iso.score_samples(X_new)

print("이상으로 잡힌 개수:", int((pred_new == -1).sum()), "/", len(new))
print()
print("가장 이상한 5개")
print(new.nsmallest(5, "score").round(4).to_string())
```

▶ 새 데이터: (40, 3) · 컬럼: ['rms', 'spectral_centroid', 'zero_crossing_rate']

▶ 이상으로 잡힌 개수: 8 / 40

▶

▶ 가장 이상한 5개

	rms	spectral_centroid	zero_crossing_rate	score
▶ 13	0.1285	2974.6	0.1216	-0.6336
▶ 31	0.0629	3028.4	0.1203	-0.5733
▶ 6	0.1229	2668.5	0.0964	-0.5579
▶ 20	0.1286	1901.8	0.0926	-0.5435
▶ 0	0.1104	2565.5	0.1073	-0.5261

개념

정답이 없는데도 8건을 골라냈습니다. 이것이 비지도학습의 핵심입니다 — 과거의 정상 패턴만 알고 있으면 처음 보는 데이터에서도 벗어남을 찾을 수 있습니다. 0-3절의 "우리 집 식구 얼굴을 알면 낯선 사람을 알아챈다"가 실제로 작동한 것입니다.

샘플	특징	왜 이상한가
13번	rms 0.1285 · 톤 2974.6	세기도 크고 톤도 높음 — 두 축 모두 벗어남
31번	rms 0.0629 (평범) · 톤 3028.4	세기는 보통인데 톤만 매우 높음 — 다변량 이상
20번	rms 0.1286 · 톤 1901.8	세기만 큼 — 명백한 이상

설비 적용

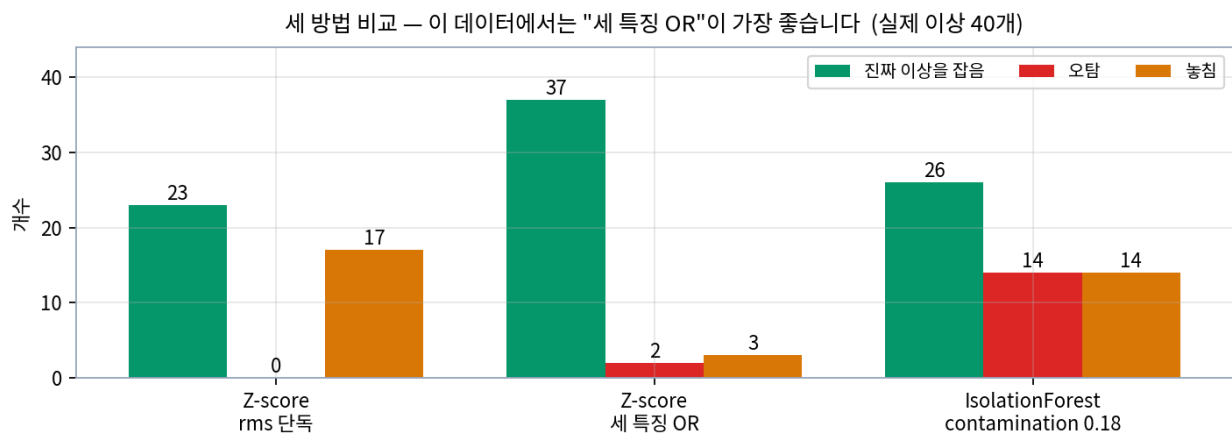
31번이 이 절의 하이라이트입니다. rms가 0.0629면 Z-score로 약 1.06이라 임계값 3은커녕 2도 못 넘습니다. Z-score였다면 통째로 놓쳤을 샘플을, IsolationForest가 **톤이 유독 높다**는 이유로 2순위로 올려 놓았습니다. 4-2 절의 다변량 이상을 실제로 잡아낸 사례입니다.

주의

새 데이터를 넣을 때 지켜야 할 두 가지가 있습니다. ① 같은 특징을 같은 순서로 넣어야 합니다 — 순서가 바뀌면 모델이 rms 자리에 톤 값을 받게 됩니다. ② 학습 때 스케일링을 썼다면 새 데이터에도 같은 기준으로 적용해야 합니다. 새 데이터로 스케일러를 다시 학습하면 기준이 흐트러집니다.

7-5. Z-score와 IsolationForest 비교 — 결과가 의외입니다

이제 오늘 배운 두 도구를 같은 데이터에 나란히 놓고 채점해 보겠습니다.



▲ 세 방법의 채점 결과 — 이 데이터에서는 "Z-score 세 특징 OR"이 진짜 이상 37건에 오탐 2건으로 가장 좋습니다

방법	탐지	진짜 이상	오탐	놓침	재현율
Z-score rms 단독	23건	23	0	17	57.5%
Z-score 세 특징 OR	39건	37	2	3	92.5%
IsolationForest 0.18	40건	26	14	14	65.0%

주의

교안이 말한 것과 결과가 다릅니다. 교안은 "Z-score 23개 → IsolationForest 27개로 개선"이라고 하는데, 그건 rms 단독 Z-score와 비교한 것입니다. 세 특징을 모두 쓴 Z-score(37건, 오탐 2건)와 비교하면 IsolationForest(26건, 오탐 14건)가 오히려 못합니다. 비교 대상을 어디에 두느냐로 결론이 뒤집힌 사례입니다.

개념

왜 이런 일이 생겼을까요. 이 데이터의 이상들은 대부분 한두 특징에서 명확하게 됩니다 — rms가 크거나, 톤이 3σ를 넘거나 합니다. 그러니 각 특징을 따로 보는 Z-score OR로도 충분히 잡힙니다. 반면 IsolationForest는 정답 비율(0.18)에 맞춰 상위 40건을 무조건 잘라내는 방식이라, 그 40건 안에 오탐 14건이 섞여 들어갑니다.

설비 적용

교훈은 "화려한 도구가 항상 정답은 아니다"입니다. 교안 마지막 장의 표현대로 특징이 한두 개고 문제가 단순하면 빠른 Z-score로 충분합니다. 게다가 Z-score는 왜 그렇게 판정했는지 설명하기가 훨씬 쉽습니다 — "rms가 표준편차 5.7배 위로 벗어났습니다"는 누구나 이해하지만, "고립 깊이가 얕습니다"는 설명이 필요합니다.

7-6. 그럼 IsolationForest는 언제 쓰나

그렇다고 IsolationForest가 쓸모없다는 뜻은 아닙니다. 이 데이터에서 안 좋았을 뿐이고, 잘 맞는 상황이 따로 있습니다.

상황	이유
여러 특징의 관계가 중요할 때	각각은 정상인데 조합이 이상한 경우 — Z-score가 원리적으로 못 하는 일
정상만 깔끔히 못 고를 때	기준선 오염에 너그러움 (5-10절) — Z-score의 치명적 약점을 피함
데이터가 크고 특징이 많을 때	거리를 계산하지 않고 분할 횟수만 세니 가벼움

실제로 오늘 데이터에서도 Z-score의 rms 단독이 놓친 17건 중 8건을 IsolationForest가 건졌습니다.

다변량 이상을 얼마나 건졌나

```
det = df["z_rms"].abs() > 3
missed = (~det) & (df.label == 1)

print("rms 단독이 놓친 실제 이상:", int(missed.sum()), "건")
print("  그중 IsolationForest가 잡은 것:", int((missed & (df.if_anom==1)).sum()),
      "건")
print("  IsolationForest도 놓친 것:", int((missed & (df.if_anom==0)).sum()), "건")
```

- ▶ rms 단독이 놓친 실제 이상: 17 건
- ▶ 그중 IsolationForest가 잡은 것: 8 건
- ▶ IsolationForest도 놓친 것: 9 건

개념

절반 가까이(8/17) 건졌습니다. 교안이 "Z-score가 놓친 다변량 이상 중 절반을 잡아냈다"고 한 것이 그대로 확인됩니다. 완벽하지는 않지만, 원리적으로 못 하던 일을 절반이나 해낸 것입니다.

7-7. 보완 관계로 쓰기 — 교차 확인

그래서 실무에서는 둘 중 하나를 고르는 것이 아니라 둘 다 돌려서 겹쳐 봅니다.

두 방법 교차표

```
zcols = ["z_rms", "z_spectral_centroid", "z_zero_crossing_rate"]
df["z_anom"] = df[zcols].abs().gt(3).any(axis=1).astype(int)
```

```
print(pd.crosstab(df["z_anom"], df["if_anom"]).to_string())
```

```

> if_anom      0      1
> z_anom
> 0          168    13
> 1           12    27

```

조합	개수	해석	다루는 법
둘 다 이상	27건	서로 다른 원리의 두 도구가 같은 결론	강한 의심 — 우선 점검
Z-score만	12건	한 특징이 크게 튀었는데 전체 관계는 평범	추가 확인 대상
IsolationForest만	13건	한 특징씩은 평범한데 조합이 특이	추가 확인 대상
둘 다 정상	168건	—	넘어감

개념

서로 다른 원리의 두 도구가 같은 결론을 내면 그만큼 믿을 만합니다. 27건은 우선 점검, 나머지 25건은 추가 확인 대상으로 두면 점검 순서가 자연스럽게 정해집니다. 33일차의 "리드타임과 우선순위" 이야기와 이어집니다.

주의

한 가지 짚어 둘 것이 있습니다. 교안은 `IsolationForest`만 잡은 것이 다변량 이상인지 확인하라*고 하는데, 실제로 확인해 보니 IF만 잡은 13건은 전부 `label=0`(정상)이었습니다. 즉 이 데이터에서는 IF 단독 탐지가 전부 오탐이었습니다. 교안의 기대와 실제 데이터가 다를 수 있다는 좋은 예이고, 그래서 항상 직접 돌려 확인해야 합니다.

7-8. 두 도구를 언제 어떻게 고르나

항목		
보는 방식	특징 하나씩	여러 특징 동시에
다변량 이상	원리적으로 놓침	일부 잡음
이상 섞인 학습	기준선 오염에 약함	어느 정도 견딜
설명하기	쉬움 — "표준편차 5.7배"	어려움 — "고립 깊이가 얕음"
설정값 부담	임계값 하나	<code>contamination</code> · <code>n_estimators</code> ·

항목		
		random_state
적합 상황	빠른 1차 점검 · 단순한 문제	관계형 이상 · 특징 많음 · 오염 데이터

개념

"무엇이 더 좋은가"가 아니라 "언제 쓰면 좋은가"가 핵심입니다. 교안 마지막 문장이 이 장을 잘 마무리합니다 — "화려한 방법을 쓰는 것 자체가 목적이 되면 본말이 뒤바뀝니다." 도구는 문제를 풀기 위해 존재하고, **문제에 맞는 도구를 고르는 것이 실력**입니다.

CHAPTER 8

이상 라벨링 (초반)

-1을 왜 굳이 바꿔야 하는가

8-1. predict 결과만으로는 부족한 두 가지

오늘 마지막 교안(26_03)은 시작만 했습니다. 강사님이 짚으신 **두 가지 문제**가 출발점입니다.

문제	내용	풀어 쓰면
① 정도를 모른다	-1과 1만으로는 얼마나 안 좋은지 판단이 안 됨	"멀쩡하냐 아니냐"만 따지니 심각도 순위를 못 매김 — 6-4절의 점수가 필요한 이유
② 형식이 다르다	우리가 쓰는 기준이 다르면 바꿔 줘야 함	MIMII는 이상이 1인데 sklearn은 -1 — 6-1절의 그 함정

개념

강사님 표현을 그대로 옮기면 이렇습니다 — "멀쩡하지 않다고 하는 걸 -1로 내둘 게 아니라, 진짜 이걸 안 좋은 것임, 안 좋다고 예측 가능했음이라고 라벨링을 붙여 줘야 한다." 즉 모델의 출력을 사람이 쓸 수 있는 꼬리표로 바꾸는 일이 라벨링입니다.

8-2. 라벨은 팀의 약속입니다

여기서 강사님이 특히 강조하신 대목이 있습니다. **라벨 표기는 기술 문제가 아니라 합의 문제**라는 것입니다.

상황	해야 할 일
원본 데이터에 기준이 있음	그 기준에 맞춥니다 — MIMII가 이상=1이면 우리도 1
아무 기준도 없음	팀이 먼저 약속을 정하고 문서화합니다
여러 명이 나눠서 분석	약속을 정하고 → 각자 코드 작성 → 그 문서의 규칙으로 합침

주의

약속을 안 정하면 무슨 일이 벌어질까요. 강사님 말씀대로 "*"코드가 막 뒤집혀서 계산을 할 수 있고 별일이 다 일어납니다.* A는 이상을 1로, B는 0으로 쓰면 두 결과를 합칠 때 **정상과 이상이 섞여 버립니다**. 그래서 **선언적인 약속을 미리 잡고 문서화**해야 합니다.

연결

33일차 CH 6의 라벨 설계와 같은 이야기입니다. 그때는 "정비 이력에서 사건을 골라 시간 구간에 1을 붙이는" 라벨링이었고, 오늘은 "모델 출력을 우리 형식으로 바꾸는" 라벨링입니다. 대상은 다르지만 **"팀이 같은 기준을 쓰기로 약속한다"**는 원칙은 똑같습니다.

8-3. 숫자 라벨과 글자 라벨

라벨을 붙이는 방법은 두 가지입니다. **숫자로 적을 수도 있고 글자로 적을 수도 있습니다**.

형태	예시	적합한 곳	컬럼명 예시
숫자 라벨	이상 = 1 · 정상 = 0	계산·비교 — 교차표, 합계, 정렬	<code>anomaly</code>
글자 라벨	"이상" · "정상"	그림·보고 — 산점도 범례에 그대로 표시	<code>label_name</code>

강사님이 **글자 라벨을 쓸 때 조심할 점 두 가지**를 짚으셨습니다.

주의점	왜 문제인가
① 계산량 증가	"이상"이라는 문자열은 숫자 1보다 훨씬 많은 자리를 차지합니다. 수만 행이 되면 메모리와 연산 시간이 늘어납니다.
② 인코딩 문제	한글을 CSV로 저장했다가 다시 읽으면 인코딩이 안 맞아 글자가 깨질 수 있습니다. 30일차 실습에서 겪었던 그 문제입니다.

개념

그래서 실무에서는 두 형태를 모두 만들어 두고 상황에 맞게 씁니다. 계산할 때는 `anomaly(0/1)`를 쓰고, 그림을 그리거나 보고서를 만들 때만 `label_name("정상"/"이상")`을 씁니다. 저장할 때는 숫자 컬럼만 남기면 인코딩 문제도 피할 수 있습니다.

주의

변환 방향을 반드시 확인하십시오. 방향이 뒤집히면 모든 결과가 거꾸로 나옵니다. 확인 방법은 6-2절에서 했던 것과 같습니다 — **변환 후 이상 개수를 세어 예상(약 40개)과 비슷한지** 보면 됩니다. 180개가 나오면 뒤집힌 것입니다.

8-4. 여기까지가 오늘 진도입니다

26_03은 총 68쪽인데 오늘은 **PART 01의 앞부분(라벨 변환의 필요성)**까지 나갔습니다. 남은 부분은 다음 시간입니다.

PART	내용	상태
01 이상 라벨링	-1/1 변환 · 원본에 붙이기 · 이상만 추려내기 · 정답과 교차표	앞부분만
02 산점도 시각화	sns.scatterplot · hue 색 구분 · 한글 폰트 · 두 특징 조합 비교	다음 시간
03 기준선과 강조	plt.axhline 기준선 · 시계열 강조 · 색 농담 · 점수 히스토그램	다음 시간
04 정비 판단 연결	탐지 → 해석 → 우선순위 → 점검 · 이상탐지 리포트 구성	다음 시간

개념

다행히 PART 02·03의 내용은 이미 절반쯤 익히신 셈입니다. 이 문서의 그림들이 전부 그 기법으로 그린 것이기 때문입니다 — 4-3절 산점도가 PART 02, 6-4절 점수 히스토그램과 6-6절 색 농담 산점도가 PART 03의 내용입니다. **다음 시간에는 그 그림을 직접 그려 보는 것이** 주 내용이 됩니다.

치트시트

오늘 나온 코드와 숫자를 한 장으로

A-1. Z-score 전체 코드

Z-score 이상탐지 — 전체

```
import pandas as pd

df = pd.read_csv("26_mimii_features.csv")
F = ["rms", "spectral_centroid", "zero_crossing_rate"]

# ① 정상만으로 기준선 → ② 전체에 식 적용
normal = df[df["label"] == 0]
for c in F:
    df["z_" + c] = (df[c] - normal[c].mean()) / normal[c].std()

# ③ 임계값 판정 (하나라도 넘으면 이상)
zcols = ["z_" + c for c in F]
df["z_anom"] = df[zcols].abs().gt(3).any(axis=1).astype(int)

print("탐지:", int(df["z_anom"].sum()), "건")
```

▶ 탐지: 39 건

A-2. IsolationForest 전체 코드

IsolationForest 이상탐지 — 전체

```
from sklearn.ensemble import IsolationForest

X = df[F]                                # label은 넣지 않음 (비지도)

iso = IsolationForest(contamination=0.18, random_state=42)  # ① 만들기
iso.fit(X)                                           # ② 학습
pred = iso.predict(X)                               # ③ 예측

df["if_anom"] = (pred == -1).astype(int)  # -1(이상) → 1로 변환
df["score"]    = iso.score_samples(X)      # 이상 점수 (작을수록 이상)

print("탐지:", int(df["if_anom"].sum()), "건")
print(pd.crosstab(df["label"], df["if_anom"]).to_string())
```

```
▶ 탐지: 40 건
▶ if_anom    0    1
▶ label
▶ 0          166  14
▶ 1           14  26
```

A-3. 오늘 쓴 도구

분류	이름	하는 일
함수	<code>pd.read_csv()</code>	CSV를 DataFrame으로 읽기
함수	<code>pd.crosstab(행, 열)</code>	두 컬럼을 교차해 개수 세기 — 채점표
메서드	<code>.mean() / .std()</code>	평균 / 표준편차
메서드	<code>.abs()</code>	절댓값 — 부호 떼기
메서드	<code>.astype(int)</code>	True/False를 1/0으로
메서드	<code>.gt(3)</code>	값보다 큰지 — DataFrame 전체에 한 번에
메서드	<code>.any(axis=1)</code>	행마다 하나라도 True가 있나
메서드	<code>.nsmallest(n, 컬럼)</code>	가장 작은 n개 — 이상 점수 정렬에
메서드	<code>.sort_values(컬럼)</code>	컬럼 기준 정렬
메서드	<code>.value_counts()</code>	값별 개수
메서드	<code>iso.fit(X)</code>	모델 학습 — label 안 넣음

분류	이름	하는 일
메서드	<code>iso.predict(X)</code>	판정 — 결과가 -1(이상) / 1(정상)
메서드	<code>iso.score_samples(X)</code>	이상 점수 — 작을수록 이상
개념	<code>StandardScaler</code>	평균 0 · 표준편차 1로 — 트리 기반엔 영향 작음
패턴	<code>df[df["label"]==0]</code>	기준선은 정상만으로 — 가장 흔한 실수

A-4. 두 도구 비교표

항목	Z-score	IsolationForest
보는 방식	특징 하나씩	여러 특징 동시에
판정 근거	평균에서 표준편차 몇 칸	몇 번 잘라야 혼자가 되나
설정값	임계값 (2 · 3 · 자유)	<code>contamination · n_estimators · random_state</code>
출력	Z-score (부호 있음)	-1/1 판정 + 점수(작을수록 이상)
다변량 이상	원리적으로 농침	일부 잡음 (오늘 8/17)
기준선 오염	탐지력 붕괴 (23→5건)	어느 정도 견뎌
설명하기	쉬움	어려움
적합 상황	빠른 1차 점검 · 단순한 문제	관계형 이상 · 특징 많음 · 오염 데이터

A-5. 오늘 검증한 숫자 전부

항목	값
데이터 크기	220행 4열 · 정상 180 · 이상 40 (18.2%)
정상 기준선 rms	평균 0.0500 · 표준편차 0.0122
정상 기준선 톤	평균 1979.0989 · 표준편차 239.1756
정상 기준선 거칠기	평균 0.0910 · 표준편차 0.0189
가장 튼 Z-score	70번 샘플 · <code>z_rms</code> 9.006
임계값 2 탐지 (rms)	36건 — 진짜 26 · 오탐 10 · 농침 14
임계값 3 탐지 (rms)	23건 — 진짜 23 · 오탐 0 · 농침 17
세 특징 OR (임계 3)	39건 — 진짜 37 · 오탐 2 · 농침 3
IsolationForest 0.18	40건 — 진짜 26 · 오탐 14 · 농침 14
IF 이상 점수 범위	-0.6785 ~ -0.3715 · 1순위 70번

항목	값
IF Top 10 적중	10개 중 9개가 실제 이상
contamination 0.05	11건 — 진짜 10 · 오탐 1 · 놓침 30
contamination 0.25	55건 — 진짜 35 · 오탐 20 · 놓침 5
StandardScaler 전후	220행 전부 동일 — 트리 기반이라 영향 없음
새 데이터 40건	8건 탐지 · 1순위 13번 · 2순위 31번(다변량)
기준선 오염 실험	표준편차 0.0122 → 0.0254 · 탐지 23건 → 5건
다변량 이상 회수	rms 단독이 놓친 17건 중 IF가 8건 회수
두 방법 교차	둘 다 이상 27건 · Z만 12건 · IF만 13건

용어·오류 사전

헷갈리는 지점과 교안과의 차이

B-1. 오늘 가장 헷갈리는 세 가지

① -1과 1의 방향

출처	이상	정상
MIMII label	1	0
sklearn predict	-1	1

주의

외우는 법: 무리 바깥(outlier)이 -1, 무리 안(inlier)이 1. 변환 코드는 `(pred == -1).astype(int)`이고, 변환 직후 개수를 세어 40 근처인지 확인하는 습관을 들이면 실수를 막을 수 있습니다.

② 이상 점수의 방향

어디서 말하는 점수	방향
개념 설명 (교안 · 5-5절)	1에 가까울수록 이상
실제 <code>score_samples()</code> / <code>decision_function()</code>	작을수록(음수 쪽) 이상

주의

개념과 구현의 부호가 반대입니다. 헷갈릴 때는 "sklearn 점수는 작을수록 이상" 하나만 외우시면 됩니다. 그래서 가장 이상한 것을 뽑을 때 `.nlargest()`가 아니라 `.nsmallest()`를 씁니다.

③ 기준선 계산 대상

✗ 이렇게 쓰면	✓ 이렇게 씁니다	결과 차이
<code>df["rms"].mean()</code>	<code>df[df.label==0]["rms"].mean()</code>	탐지 23건 → 5건으로 붕괴

주의

IsolationForest에는 이 구분이 없습니다. `fit(X)`에 전체를 넣습니다 — 평균에 의존하지 않아 오염에 너그럽기 때문입니다(5-10절). **Z-score**에서만 반드시 정상을 골라내야 합니다.

B-2. 교안 숫자와 실제 결과의 차이

교안에 적힌 예상 결과와 저희가 실제로 돌린 결과가 몇 군데 다릅니다. **틀렸다가보다는 환경 차이**이지만, 알고 계시면 당황하지 않으실 것입니다.

항목	교안	실제 실행	왜 다른가
IF가 잡은 진짜 이상	약 27개	26개	경계선 샘플 한 건이 사이킷런 버전에 따라 갈림
교차표 정상 맞힘	167건	166건	위와 같은 이유 — 한 건 차이
숨은 이상 개수	16개	17개	137번(<code>z_rms</code> 2.976)은 다변량이 아니라 경계선 케이스
실습 8 놓침	17건	17건 — 일치	—

개념

"약 27개"라고 적힌 것은 그 자체로 좋은 표기입니다. 무작위가 들어간 알고리즘이라 환경에 따라 한두 건 달라질 수 있고, 교안도 그것을 알고 "약"을 붙인 것입니다. `random_state`를 고정해도 사이킷런 버전이 다르면 결과가 미세하게 달라질 수 있습니다.

B-3. 교안 기대와 데이터가 다른 지점

교안 26_02 마지막 실습의 체크포인트 중 하나가 **실제 데이터와 맞지 않았습니**다. 짚어 두겠습니다.

교안 체크포인트	실제 확인 결과
"IsolationForest만 잡은 것이 세기 보통, 툰·거칠기 높은 이상인가"	아닙니다 — IF만 잡은 13건은 전부 label=0(정상)이었습니다

주의

IF만 잡은 13건이 전부 오탐이었습니다. 다변량 이상을 잡아 주기를 기대한 자리인데 실제로는 그렇지 않았습니다. 다만 IF가 다변량 이상을 아예 못 잡은 것은 아닙니다 — 7-6절에서 확인했듯 `rms` 단독이 놓친 17건 중 8건을 건졌는데, 그 8건은 **Z-score** 세 특징 **OR도 함께 잡은 것들**이라 "둘 다 이상" 칸에 들어갔습니다.

개념

이런 차이를 발견하는 것이 오히려 좋은 학습입니다. 교안은 일반적인 설명을 하고, 데이터는 구체적인 사실을 말합니다. 직접 둘러서 확인하지 않으면 알 수 없는 것이고, 실무에서도 마찬가지입니다.

B-4. 헛갈리기 쉬운 용어 쌍

용어 A	용어 B	차이
지도학습	비지도학습	정답을 주고 배움 / 정답 없이 구조만으로
평균	표준편차	중심이 어디인가 / 평소 얼마나 흔들리는가
Z-score	임계값	몇 칸 떨어졌나(계산) / 몇 칸부터 이상인가(우리가 정함)
거짓 경보	놓침	정상인데 이상이라 함 / 이상인데 정상이라 함
재현율	정밀도	실제 이상 중 잡은 비율 / 잡은 것 중 진짜 비율
특징값	원본 신호	수만 개를 요약한 대표 숫자 / 원래의 소리 파형
rms	spectral_centroid	소리 세기(볼륨) / 소리 톤(무게중심)
다변량 이상	명백한 이상	각각 정상인데 조합이 이상 / 한 특징이 크게 튼
고림 깊이	이상 점수	혼자 되기까지 분할 횟수 / 그 깊이를 환산한 값
contamination	n_estimators	이상으로 볼 비율 / 나무 개수(안정성)
fit	predict	모델을 만드는 학습 / 만든 모델로 판정
회색 지대	명백한 구간	정상·이상이 겹쳐 어떤 도구도 헛갈림 / 확실히 갈림

개념

"Z-score와 임계값"의 구분이 특히 중요합니다. Z-score는 데이터가 정해 주는 계산 결과이고, 임계값은 우리가 정하는 정책입니다. 이 둘을 섞으면 "통계가 3이라고 했다"는 잘못된 말이 나옵니다 — 통계는 아무것도 정해 주지 않았고 우리가 3을 골랐을 뿐입니다.

하이라이트와 다음 시간

오늘의 핵심 발견 · 다음 진도

C-1. 오늘의 발견 네 가지

① 한 축에서 숨은 이상이 두 축에서는 보입니다

rms만 보면 17건이 정상 속에 파묻혀 있었는데, 톤과 거칠기를 축으로 놓으니 **자기들끼리 뚜렷한 무리**를 이루고 있었습니다. **이상이 없었던 것이 아니라 우리가 잘못된 축으로 보고 있었던 것입니다.**

보는 축	놓친 이상 17건의 위치
rms 하나	정상 무리 한가운데 (z 0.4~2.1) — 안 보임
spectral_centroid × zero_crossing_rate	오른쪽 위에 별도 무리 — 확연히 보임

설비 적용

어떤 두 특징을 고르느냐가 결정적입니다. 세기 이상은 rms가 들어간 조합에서, 다변량 이상은 rms를 뺀 조합에서 잘 보입니다. 그래서 산점도는 **한 장만 그리지 말고 조합을 바꿔 여러 장 그려** 봐야 합니다.

② 기준선을 잘못 잡으면 탐지력이 무너집니다

기준선	rms 표준편차	값 0.120의 Z	임계 3 탐지
정상만	0.0122	5.74	23건
이상 섞임	0.0254	2.41	5건

개념

코드 한 글자 차이로 탐지가 23건에서 5건이 됐습니다. 명백한 이상조차 Z가 2.41로 떨어져 임계값을 못 넘습니다. 알고리즘을 아무리 잘 골라도 **입력이 오염되면 방법이 없다**는 것을 보여 주는 사례입니다.

③ 이 데이터에서는 Z-score가 더 좋았습니다

교안의 서사는 "Z-score의 한계 → IsolationForest로 개선"인데, 세 특징을 다 쓴 **Z-score**와 비교하면 결과가 뒤집힙니다.

방법	진짜 이상	오탐	놓침	재현율
Z-score 세 특징 OR	37	2	3	92.5%
IsolationForest 0.18	26	14	14	65.0%

개념

교안이 비교한 것은 rms 단독 Z-score(23건)였습니다. 비교 대상을 어디에 두느냐로 결론이 완전히 달라진 사례입니다. 보고서를 쓸 때 "무엇과 비교한 개선인지"를 밝히는 것이 왜 중요한지 보여 줍니다.

설비 적용

그렇다고 IsolationForest가 나쁜 도구는 아닙니다. 이 데이터의 이상들이 대부분 한두 특징에서 명확히 튀어서 Z-score OR로 충분했을 뿐입니다. 특징이 열 개가 넘고 관계가 복잡한 데이터였다면 결과가 반대로 나왔을 것입니다. 교안 마지막 문장대로 문제에 맞는 도구를 고르는 것이 핵심입니다.

④ 스케일링이 아무 효과가 없었습니다

값 범위가 0.05 · 1979 · 0.091로 제각각인데도, StandardScaler를 적용한 결과가 220행 전부 원본과 똑같았습니다.

개념

트리는 "이 값보다 큰가 작은가"만 묻기 때문입니다. 스케일링은 값을 옮기지만 크기 순서는 바꾸지 않으므로, 트리 입장에서는 자를 자리가 똑같습니다. 다만 거리를 직접 재는 방법(K-평균 등)에서는 거의 필수이니, "트리 기반에만 영향이 작다"로 기억해 두시면 됩니다.

주의

교훈은 "좋다고 알려진 전처리도 확인하고 쓰라"입니다. 무작정 넣으면 시간만 쓰고 효과가 없을 수 있습니다. 오늘처럼 적용 전후를 나란히 돌려 비교하는 습관이 정석입니다.

C-2. 저녁에 해 보면 좋을 것

난이도	과제	확인 포인트
쉬움	임계값을 2.5로 바꿔 탐지 개수와 오탐을 확인	시소가 어느 지점에서 균형인지
쉬움	contamination을 0.15와 0.20으로 돌려 비교	0.18이 정말 최선인지
보통	spectral_centroid와 zero_crossing_rate 두 개만으로 IsolationForest 학습	rms를 빼면 다변량 이상을 더 잘 잡는지
보통	오늘 그림 9장 중 하나를 직접 그려 보기	26_03 PART 02의 예습이 됩니다

난이도	과제	확인 포인트
도전	random_state를 42가 아닌 값 다섯 개로 바꿔 결과 편차 확인	무작위가 결과에 얼마나 영향을 주는지
도전	두 방법 결과를 합쳐 "강한 의심 / 추가 확인" 목록 만들기	7-7절 교차표를 실제 목록으로

설비 적용

세 번째 과제가 특히 흥미로울 것입니다. rms를 빼면 IsolationForest가 다변량 이상에만 집중하게 되는데, 이 데이터에서는 성능이 오를 가능성이 있습니다. 어떤 특징을 넣고 뺄지도 하나의 설계 결정이라는 감각을 얻을 수 있습니다.

C-3. 다음 시간 예고

교안	내용	오늘과의 연결
26_03 PART 01 뒷부분	라벨을 원본에 붙이기 · 이상만 추려내기 · 정답과 교차표	오늘 6-2·6-3절에서 이미 절반 익힘
26_03 PART 02	산점도 시각화 — hue 색 구분 · 한글 폰트 · 조합 비교	오늘 4-3·6-6절 그림을 직접 그리기
26_03 PART 03	기준선(axhline) · 시계열 강조 · 색 농담 · 점수 히스토그램	오늘 6-4절 히스토그램을 직접 그리기
26_03 PART 04	정비 판단 연결 — 탐지→해석→우선순위→점검 · 리포트	33일차 성과 평가와 이어짐

연결

PART 04가 33일차와 직접 이어집니다. "이상탐지가 잡은 것은 고장이 아니라 평소와 다름"이라는 문장이 나오는데, 이것은 33일차의 "점검을 권합니다" 표현과 같은 이야기입니다. 그리고 거짓 경보와 농침의 균형을 설비 중요도에 따라 다르게 잡는 것도 33일차 CH 5의 임계값 조정과 똑같습니다.

EPILOGUE

맺음말

오늘의 성취와 복습 루틴

오늘 하신 것

구간	성취
CH 1	기준선을 세우고 Z-score를 계산하는 법, 그리고 임계값이 통계가 아니라 우리가 정하는 정책이라는 것을 배웠습니다
CH 2	소리를 세 특징으로 요약한다는 발상과, 각 특징이 무엇을 잡아내는지 알게 되었습니다
CH 3	기준선 → 계산 → 판정의 세 단계를 코드로 실행하고, 결과를 정비 문장으로 옮겼습니다
CH 4	Z-score의 세 한계를 숫자로 확인하고, 왜 다음 도구가 필요한지 이해했습니다
CH 5~6	scikit-learn의 만들기-fit-predict 흐름을 익히고, -1/1의 함정과 이상 점수를 다뤘습니다
CH 7	설정값을 바꿔 실험하고, 두 도구를 나란히 채점해 각자의 자리를 확인했습니다
CH 8	모델 출력을 사람이 쓸 수 있는 라벨로 바꾸는 일이 왜 팀의 약속인지 배웠습니다

복습 루틴 제안

시점	할 것	걸리는 시간
오늘 밤	A-1·A-2 전체 코드 두 개만 다시 읽기	5분
내일 아침	B-1의 헛갈리는 세 가지를 손으로 써 보기	7분

주의

오늘 자료 중 가장 오래 쓰이는 것은 A-1의 세 줄입니다 — 정상만 골라 기준선, 전체에 식 적용, 임계값 판정. 이 흐름은 Z-score만이 아니라 거의 모든 통계 기반 이상탐지에 그대로 적용됩니다. 그리고 그 앞에 `df[df.label==0]`이 붙어 있다는 것을 잊지 마십시오.

한 문장으로 줄이면 이렇습니다. **정상을 정확히 알면 이상은 저절로 드러납니다.** 오늘의 두 도구는 그 "아는 법"이 달랐을 뿐입니다 — 하나는 중심에서의 거리로, 다른 하나는 고립되는 속도로.

수고하셨습니다.